

Eduardo Thomaz dos Santos e Eduardo Niza Minosso

Sistema de Gerenciamento de Bancos de Dados Chave-Valor Distribuído

São Paulo, SP

2024

Eduardo Thomaz dos Santos e Eduardo Niza Minosso

Sistema de Gerenciamento de Bancos de Dados Chave-Valor Distribuído

Trabalho de conclusão de curso apresentado
ao Departamento de Engenharia de Computa-
ção e Sistemas Digitais da Escola Politécnica
da Universidade de São Paulo para obtenção
do Título de Engenheiro.

Universidade de São Paulo – USP

Escola Politécnica

Departamento de Engenharia de Computação e Sistemas Digitais (PCS)

Orientador: Prof. Dr. Jorge Rady

São Paulo, SP

2024

Autorizo a reprodução e divulgação total ou parcial deste trabalho, por qualquer meio convencional ou eletrônico, para fins de estudo e pesquisa, desde que citada a fonte.

Catálogo-na-publicação

Minosso, Eduardo

Sistema de Gerenciamento de Banco de Dados Distribuído / E. Minosso,
E. dos Santos -- São Paulo, 2024.

73 p.

Trabalho de Formatura - Escola Politécnica da Universidade de São
Paulo. Departamento de Engenharia de Computação e Sistemas Digitais.

1.Banco de Dados 2.Sistemas Distribuídos I.Universidade de São Paulo.
Escola Politécnica. Departamento de Engenharia de Computação e Sistemas
Digitais II.t. III.dos Santos, Eduardo

*Este trabalho é dedicado às crianças adultas que,
quando pequenas, sonharam em se tornar cientistas.*

Agradecimentos

Os agradecimentos principais são direcionados a todos que nos ajudaram direta, ou indiretamente para a conclusão do curso. As nossas famílias, por todo o apoio e ajuda por toda a nossa graduação. Aos nossos amigos que fizemos durante esse curso, que estiveram sempre ao nosso lado e nos incentivaram nos momentos difíceis. E aos professores que nos ensinaram muito, corrigindo e guiando para que nós pudessemos nos formar como Engenheiros de Computação.

Resumo

Este trabalho apresenta o desenvolvimento de um *Sistema de Gerenciamento de Bancos de Dados Chave-Valor Distribuído*, criado para lidar com grandes volumes de dados de maneira escalável e consistente. Com base em conceitos de sistemas distribuídos e algoritmos de consenso, a solução proposta demonstra como um banco de dados chave-valor pode garantir alta disponibilidade em leituras, manter a integridade de dados binários (como arquivos PDF) e oferecer mecanismos para recuperação de falhas por meio de replicação e eleições de líderes. Ao longo do projeto, são abordados desafios práticos relacionados à limitação de taxa de inserções e mecanismos de observabilidade, evidenciando oportunidades de evolução e aplicabilidade em cenários reais de alta demanda.

Palavras-chave: Sistemas distribuídos; Banco de dados; Consenso; Chave-valor; Escalabilidade.

Resumo

This work presents the development of a *Distributed Key-Value Database Management System* designed to handle large volumes of data in a scalable and consistent manner. Based on concepts from distributed systems and consensus algorithms, the proposed solution demonstrates how a key-value database can ensure high availability for read operations, maintain the integrity of binary data (such as PDF files), and offer mechanisms for fault recovery through replication and leader elections. Throughout the project, practical challenges related to insertion rate limitations and observability mechanisms are addressed, highlighting opportunities for improvement and applicability in real-world high-demand scenarios.

Keywords: Distributed systems; Database; Consensus; Key-value; Scalability.

Lista de ilustrações

Figura 1 – Teorema CAP representado visualmente.	16
Figura 2 – Exemplo de escalonamento utilizando o algoritmo de Round Robin. . .	17
Figura 3 – Requisição considerando um nó único.	19
Figura 4 – Requisição considerando três nós que precisam chegar a um consenso. .	20
Figura 5 – Visão geral dos componentes e suas interdependências.	23
Figura 6 – Fluxo das requisições no sistema.	34
Figura 7 – Exemplo de inicialização com três instâncias iniciais e duas dinâmicas. .	35
Figura 8 – Exemplo de requisição POST para salvar um valor extenso no sistema. .	54
Figura 9 – Contagem de 2.106.540 caracteres do <code>value</code> , conforme verificado no Notepad.	55
Figura 10 – Valor recuperado do sistema via GET, confirmando a persistência. . . .	56
Figura 11 – Atualização via HTTP PUT para a chave “arquivo-gigante”.	56
Figura 12 – Nova requisição HTTP GET, demonstrando o valor atualizado.	57
Figura 13 – Remoção do registro via HTTP DELETE para a chave “arquivo-gigante”. .	57
Figura 14 – Tentativa de HTTP GET após a remoção, evidenciando que o valor não existe mais.	58
Figura 15 – Logs da aplicação ao subir o cluster e iniciar o processo de eleição. . .	58
Figura 16 – Terminal do serviço durante uma nova eleição, exibindo paradas de nós e reeleições.	59
Figura 17 – Terminal exibindo a execução de um <code>script</code> em Python que envia requisições POST ao sistema.	60

Figura 18 – Logs internos do serviço, evidenciando o recebimento das requisições e o retorno de 201 Created	61
Figura 19 – 500 Internal Server Error retornado pelo servidor durante a sobrecarga de requisições.	62
Figura 20 – Amostra do terminal indicando 200 OK para 1000 requisições GET em menos de 2 segundos, sem erros no lado do servidor.	63
Figura 21 – Trecho do terminal mostrando correspondência exata entre a chave solicitada e a chave retornada em cada requisição.	65
Figura 22 – Registro de PDF na aplicação web, atribuindo o nome “ <i>historico escolar</i> ” ao arquivo <code>historicoescolarListar (28).pdf</code> antes de pressionar “Registrar PDF”.	66
Figura 23 – Visualização da aplicação web ao recuperar o PDF. O campo “ nome do PDF desejado ” contém “ historico escolar ”, e o botão “ BAIXAR PDF ” foi acionado. A aba <i>Networking</i> do navegador exibe a resposta em Base64 , confirmando o download bem-sucedido do arquivo.	66

Sumário

1	INTRODUÇÃO	11
1.1	Motivação	11
1.2	Objetivos	12
1.3	Justificativa	12
1.4	Organização do Trabalho	13
2	ASPECTOS CONCEITUAIS	15
2.1	Propriedades de Sistemas de Banco de Dados	15
2.2	Consenso entre nós	15
2.2.1	Justificativa para Escolha do Raft	16
2.3	Balanceamento de Carga	17
2.4	Arquitetura	17
2.4.1	Descrição da arquitetura com base em suas camadas	18
2.4.2	Descrição das aplicações do sistema	20
2.4.3	Descrição dos componentes da arquitetura	21
2.5	Observabilidade	23
2.6	Banco de Dados Chave-Valor	24
2.6.1	Aplicações que Utilizam Bancos de Dados Chave-Valor	24
3	MÉTODO DO TRABALHO	26
3.1	Estudo do tema	26
3.2	Especificação de Requisitos	26
3.3	Planejamento do Projeto	26
3.4	Implementação	26
3.5	Testes	27
4	ESPECIFICAÇÃO DE REQUISITOS	28
4.1	Requisitos Funcionais	28
4.1.1	RF1 – Salvar Dados Binários	28
4.1.2	RF2 – Ler Dados com Base em uma Chave	28
4.1.3	RF3 – Deletar Dados com Base em uma Chave	29
4.1.4	RF4 – Atualizar Dados com Base em uma Chave	29
4.2	Requisitos Não Funcionais	29
4.2.1	RNF1 – Escalabilidade Horizontal	29
4.2.2	RNF2 – Consistência dos Dados	29
4.2.3	RNF3 – Alta Performance	29

4.2.4	RNF4 – Tolerância a Falhas	30
4.2.5	RNF5 – Observabilidade	30
4.3	Considerações Finais	30
5	DESENVOLVIMENTO DO TRABALHO	31
5.1	Tecnologias Utilizadas	31
5.1.1	Linguagem de Programação Go	31
5.1.2	Protocol Buffers (Protobuf)	31
5.1.3	gRPC	32
5.1.4	Gerenciamento de Configuração com YAML	32
5.1.5	Gerenciamento de Estado e Persistência	32
5.1.6	Controle de Concorrência com sync e Goroutines	32
5.1.7	Logger para Monitoramento	33
5.1.8	Outras Ferramentas	33
5.2	Aplicações Desenvolvidas	34
5.2.1	CLI - Aplicações Core	34
5.2.1.1	Orquestrador da Aplicação	36
5.2.1.2	API REST e Balanceador de Carga	36
5.2.1.3	Servidores gRPC	36
5.2.2	Interface Web e Integração com o Sistema	37
5.2.2.1	Principais Funcionalidades da Interface Web	37
5.2.2.2	Recuperação de Arquivos PDF	37
5.2.2.3	Funcionamento Detalhado	39
5.3	Implementação e Algoritmos	39
5.3.1	Implementação do Algoritmo Raft	40
5.3.1.1	Estrutura Geral do Nó Raft	40
5.3.2	Implementação do Servidor e Inicialização do Cluster	40
5.3.2.1	Servidor gRPC Integrado ao Raft	40
5.3.2.2	Inicialização e Configuração do Cluster	41
5.3.2.2.1	Inicialização Estática	41
5.3.2.2.2	Inicialização Dinâmica via Arquivo de Configuração	42
5.3.2.3	Execução e Gerenciamento do Cluster	42
5.3.2.4	Integração com o Algoritmo Raft e o Sistema	43
5.3.2.5	Benefícios para o Projeto	43
5.3.2.6	Estados do Nó	44
5.3.2.7	Persistência de Estado	44
5.3.2.8	Mecanismo de Eleições	45
5.3.2.9	Replicação de Logs e <i>Heartbeats</i>	45
5.3.2.10	Aplicação de Entradas de Log	46
5.3.2.11	Benefícios da Implementação	47

5.3.3	Definição das Interfaces com Protocol Buffers	47
5.3.3.0.1	Arquivo <code>raft.proto</code>	47
5.3.3.1	Geração de Código com <i>protoc</i>	49
5.3.3.2	Integração interna	49
5.3.3.2.1	Implementação do Servidor	49
5.3.3.2.2	Implementação do Cliente	49
5.3.3.3	Benefícios da Utilização do Protobuf	49
5.3.4	Módulo <i>grpcutil</i>	50
5.3.4.1	Cliente gRPC	50
5.3.4.2	Configuração do Cliente	50
5.3.4.3	Cliente Mock	51
5.4	Testes e Avaliação	51
5.4.1	Estratégia de Teste	51
5.4.2	Plano de Testes	53
5.4.2.1	Testes Funcionais do Banco de Dados	53
5.4.2.2	Testes de Consistência e Eleição (Raft)	57
5.4.2.3	Testes de Consistência de Logs	60
5.4.2.4	Limitação de Taxa e Erros em Altas Taxas de Inserção	61
5.4.2.5	Teste de Carga com 1000 Requisições GET	62
5.4.2.6	Teste de Inserção e Recuperação de PDF via Interface Web	64
6	CONSIDERAÇÕES FINAIS	67
6.1	Análise Geral do Desempenho e da Escalabilidade	67
6.2	Desafios encontrados	68
6.3	Validação de Integridade e Tolerância a Falhas	68
6.4	Conclusão Geral	69
6.5	Perspectivas de Continuidade	69
	REFERÊNCIAS	71

1 Introdução

1.1 Motivação

Nas últimas décadas, a internet surgiu e seu uso cresceu exponencialmente. Com isso, a necessidade de lidar com grandes volumes de dados aumentou a demanda por recursos computacionais, tornando os sistemas tradicionais, centralizados e baseados em uma única máquina, insuficientes para muitas tarefas. A computação em nuvem também emergiu, possibilitando a flexibilização do uso de máquinas conforme a necessidade de carga, reduzindo custos e investimentos em infraestrutura. Diante da crescente demanda por aplicações que fazem uso intenso de dados e da facilidade em escalar horizontalmente, a pesquisa sobre sistemas distribuídos tem se tornado fundamental para garantir alta disponibilidade, resiliência e desempenho eficiente em ambientes computacionais modernos. Este contexto ressalta a importância de desenvolver soluções inovadoras em bancos de dados distribuídos, capazes de atender às exigências de escalabilidade e confiabilidade impostas pelas aplicações contemporâneas.

Sistemas distribuídos funcionam através de uma rede de aplicações interconectadas que trabalham em busca de um mesmo objetivo. Nestes sistemas, o trabalho é dividido entre as diversas máquinas que o compõem, resultando em melhor uso dos recursos computacionais e performance. Além da performance, também há a vantagem em casos de falha, pois quando um computador falha, a carga pode ser transferida para outro sistema. Diante disso, surge o conceito de escalabilidade horizontal, onde ao invés de aumentar recursos da máquina em que a aplicação roda, como CPU e memória, busca-se aumentar o número de réplicas da aplicação.

Sistemas de gerenciamento de bancos de dados representam uma categoria de aplicação amplamente beneficiada por sistemas distribuídos, especialmente em cenários que demandam alta disponibilidade e desempenho escalável. Grandes SGBDs, como Postgres, MySQL, MongoDB e Redis, possuem suporte para escalabilidade horizontal, que pode ser alcançada de duas formas principais: o aumento do número de réplicas dentro do serviço e o particionamento dos dados. O uso de réplicas permite lidar com um maior volume de acessos simultâneos, melhorando a capacidade de leitura e a resiliência do sistema em casos de falhas. Já o particionamento divide os dados entre diferentes instâncias, distribuindo a carga de trabalho e otimizando o desempenho de operações complexas. Esses conceitos estão fortemente alinhados com a arquitetura de microserviços, na qual bancos de dados distribuídos são frequentemente empregados para atender às demandas específicas de diferentes serviços, permitindo escalabilidade e maior autonomia na gestão de dados ([DECANDIA et al., 2007](#)).

Além dos ganhos de performance, outro benefício é a tolerância a falhas de instâncias. Se tratando de sistemas de armazenamento, que muitas vezes usam discos rígidos mecânicos devido ao baixo custo, as falhas podem ser catastróficas e resultar em uma perda de dados importantes. No entanto, através da replicação de dados, caso ocorra uma falha em um dispositivo, é possível usar outros dispositivos como backup e recuperar os dados sem comprometer o funcionamento do sistema.

Um dos primeiros exemplos reais onde a computação distribuída teve um grande impacto comercialmente foi na criação do Google File System (GFS) [1]. No GFS, além de ganhos significativos em performance, o seu sistema de replicação também possibilitou se recuperar de falhas em sistemas de armazenamento, através da replicação dos dados entre diferentes máquinas. Assim, uma falha em um HD não impactava na perda dos dados.

1.2 Objetivos

O objetivo deste projeto é desenvolver um sistema de banco de dados distribuído baseado no modelo chave-valor, que seja capaz de lidar com um grande volume de requisições, se recuperar de falhas nos computadores e que possa ser escalado horizontalmente conforme a necessidade. O

Para isto, o sistema será composto por múltiplas replicações, onde cada uma armazenará cópias dos dados e será responsável pela replicação em outros nós. Para garantir a consistência dos dados entre esses múltiplos nós, será utilizado um algoritmo de consenso que permitirá a resolução de conflitos através da eleição de nós como líderes.

1.3 Justificativa

Neste contexto, uma aplicação de banco de dados no modelo chave-valor pode proporcionar através da escalabilidade horizontal uma tolerância a falhas devido à forma como os dados são particionados e replicados entre diversos nós da rede. Por exemplo, o sistema *Cassandra* (LAKSHMAN; MALIK, 2010), originalmente desenvolvido pelo Facebook, demonstrou que a adoção de uma topologia *ring* (anel) e a distribuição dos dados em múltiplos nós permitem lidar com falhas de maneira transparente, bem como adicionar ou remover nós sem interromper as operações de leitura e escrita. Isso viabiliza escalabilidade linear à medida que o volume de dados cresce.

Além disso, o *Bigtable* (CHANG et al., 2008), criado pelo Google, mostrou como o particionamento baseado em colunas e a replicação controlada podem aumentar significativamente a capacidade de processamento de grandes conjuntos de dados distribuídos em datacenters geograficamente dispersos. Essa arquitetura inspirou outros sistemas de banco

de dados chave-valor, apontando para estratégias de fragmentação (sharding) e replicação como pilares essenciais para lidar com requisitos de latência e disponibilidade.

Diante dessas referências, o presente projeto busca aproveitar princípios semelhantes: ao implementar um banco de dados chave-valor distribuído, adotamos estratégias de particionamento e replicação para garantir que o sistema possa escalar horizontalmente e manter a consistência dos dados mesmo em cenários de falha de nós. Assim, baseamo-nos nas lições aprendidas por [Lakshman e Malik \(2010\)](#) e [Chang et al. \(2008\)](#) para estruturar a nossa abordagem de distribuição e garantir resiliência, conectando diretamente esses conceitos às necessidades de aplicações modernas intensivas em dados.

Por fim, a escolha de um modelo chave-valor para este gerenciador de banco de dados foi feita por conta da sua simplicidade de uso, com poucas operações e um fácil aprendizado, e, ao mesmo tempo, flexível, podendo ser usado em diferentes contextos por outras aplicações, por exemplo, a criação de um gerenciador de bancos de dados mais complexo, como um relacional.

1.4 Organização do Trabalho

Este documento apresentará os aspectos conceituais relevantes para o desenvolvimento de sistemas gerenciadores de banco de dados distribuídos no Capítulo 2. Em seguida, no Capítulo 3, apresentará a metodologia utilizada para o desenvolvimento do trabalho. No Capítulo 4 apresentará a especificação de requisitos do projeto.

Estrutura do Trabalho

- **Capítulo 1 - Introdução:** Apresenta a ideia geral do trabalho, seus objetivos, motivações, justificativas e a estrutura em que está organizado.
- **Capítulo 2 - Aspectos Conceituais:** Apresenta definições de conceitos essenciais para o entendimento do projeto através da leitura desta monografia.
- **Capítulo 3 - Método do Trabalho:** Descreve as etapas adotadas para o desenvolvimento e organização do projeto.
- **Capítulo 4 - Especificação de Requisitos:** Apresenta os requisitos funcionais e não funcionais levantados no planejamento do projeto, que serão contemplados nos resultados.
- **Capítulo 5 - Desenvolvimento do Trabalho:** Relata o processo de desenvolvimento da aplicação, as tecnologias utilizadas e os desafios enfrentados. Também inclui os processos de testes, validações e seus resultados.

- **Capítulo 6 - Considerações Finais:** Apresenta as conclusões obtidas ao final do trabalho, as contribuições realizadas e possíveis direções para trabalhos futuros.

2 Aspectos Conceituais

2.1 Propriedades de Sistemas de Banco de Dados

Uma das formas mais comuns de definir as propriedades desejáveis das transações de banco de dados é através da sigla ACID (Atomicidade, Consistência, Isolamento e Durabilidade) ([MEDJAHED](#); [OUZZANI](#); [ELMAGARMID, 2009](#)). Estas propriedades definem o seguinte:

- **Atomicidade:** Esta propriedade define que uma transação não pode ser parcialmente completada, então garante que cada transação é tratada de forma unitária, seja falhando completamente ou sendo feita completamente com sucesso.
- **Consistência:** Esta propriedade define que o banco de dados só pode transicionar de um estado consistente para outro, então qualquer dado escrito no banco de dados deve ser válido de acordo com todas as regras.
- **Isolamento:** Esta propriedade garante o funcionamento adequado do sistema em caso de transações concorrentes.
- **Durabilidade:** Esta propriedade garante que caso uma transação tenha sido realizada, os dados serão persistidos mesmo em caso de falhas (com por exemplo, falta de energia elétrica nas máquinas em que o sistema é executado).

Apesar deste sistema não implementar transações diretamente, permitindo apenas operações unitárias e isoladas deseja-se aproximar as características dele para que atenda a requisitos como durabilidade, isolamento e consistência.

2.2 Consenso entre nós

Em 1999, Eric Brewer e Armando Fox publicaram um artigo ([BREWER](#); [FOX, 1999](#)) onde introduziam o teorema CAP que demonstra que é impossível que um sistema possua três características simultâneas: consistência forte, alta disponibilidade e tolerância a particionamentos. Tendo, portanto, que escolher entre duas das características, conforme ilustrado na figura 1.

Conforme discutido na seção anterior, para este sistema, a consistência dos dados é importante e deve-se garantir que os nós concordem nos dados que são salvos no sistema sempre. Para fins de escalabilidade e performance neste projeto, a consistência forte

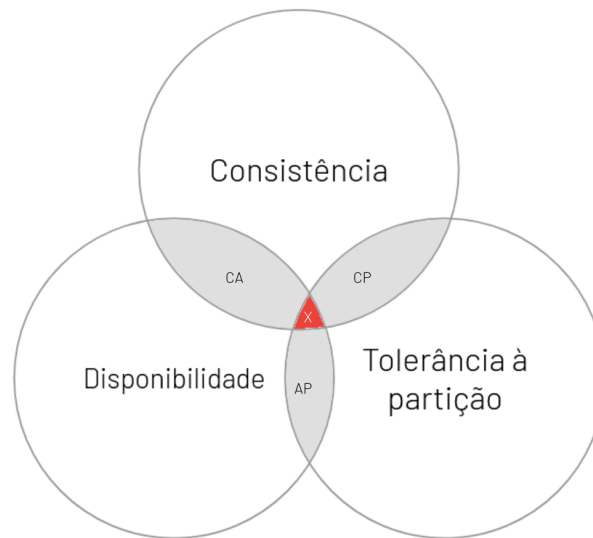


Figura 1 – Teorema CAP representado visualmente.

e tolerância a particionamentos será priorizada em detrimento de alta disponibilidade. Em casos de particionamento na rede, alguns nós podem se tornar temporariamente indisponíveis.

Já existem algoritmos que proporcionam a possibilidade de ter forte consistência com tolerância a particionamento. Os principais exemplos são Paxos e Raft ([ONGARO; OUSTERHOUT, 2014](#)) e Zab (utilizado na ferramenta Apache Zookeeper) ([HUNT et al., 2010](#)).

2.2.1 Justificativa para Escolha do Raft

O algoritmo Raft foi selecionado em detrimento de outros protocolos de consenso, como Paxos e Zab, por combinar *robustez* e *simplicidade* na implementação. De acordo com Ongaro e Ousterhout ([ONGARO; OUSTERHOUT, 2014](#)), o Raft foi projetado para ser mais compreensível que Paxos, simplificando aspectos críticos como a eleição de líder e a replicação de *logs*. Em seu trabalho seminal, Lamport ([LAMPORT, 2005](#)) apresenta o Paxos como um protocolo confiável, mas que demanda certo grau de complexidade extra na implementação e na depuração do código. Já o Zab, empregado no Apache ZooKeeper ([HUNT et al., 2010](#)), também oferece propriedades de consistência forte e tolerância a falhas, porém, sua documentação e sua comunidade de suporte não são tão difundidas quanto as do Raft.

Em síntese, a equipe enxerga clareza na arquitetura do Raft (*líder e seguidores*). O artigo de Ongaro e Ousterhout ([ONGARO; OUSTERHOUT, 2014](#)) aprofunda a justificativa de que o Raft facilita o entendimento e a manutenção do código, fator decisivo em um projeto acadêmico que busca minimizar riscos de inconsistências e maximizar a confiabilidade do sistema distribuído. Logo, o Raft atende aos requisitos de *consistência*

forte e tolerância a partições (essenciais para o objetivo deste trabalho), ao mesmo tempo em que reduz a complexidade de implementação observada em outras abordagens clássicas como Paxos e Zab.

2.3 Balanceamento de Carga

Além de distribuir os nós, o sistema deve também conseguir balancear a carga recebida.

Dada a forma como algoritmo Raft funciona, todas as requisições que alteram dados, como salvar e deletar dados, devem ser realizadas diretamente pelo líder, enquanto que requisições de leitura podem ser realizadas por qualquer nó.

Por isso, o grupo optou por criar seu próprio balanceador na aplicação, ao invés de utilizar soluções disponíveis no mercado (como o NGINX). Este componente da aplicação tem o conhecimento de qual é o líder dos nós no momento atual para direcionar certas requisições a estes.

Para as outras requisições, como de leitura de dados, que podem ser feitas por qualquer nó, o balanceador utiliza o algoritmo round-robin, utilizado em escalonamento de processos (STALLINGS, 2015). Este algoritmo transfere a carga para um nó de cada vez, seguindo sempre uma mesma ordem, conforme representado na figura 2.

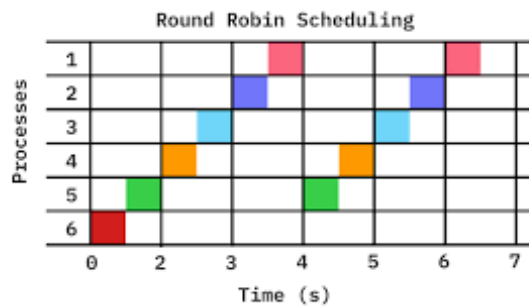


Figura 2 – Exemplo de escalonamento utilizando o algoritmo de Round Robin.

2.4 Arquitetura

O projeto adota uma arquitetura distribuída em camadas, combinando simplicidade de integração com eficiência operacional. Na camada externa, foi escolhida uma interface baseada em **HTTP REST**, devido à ampla adoção do protocolo e à sua compatibilidade com navegadores e serviços web. Para o uso do gRPC em aplicações Web é necessário recorrer a fontes informais como o gRPC for Web Clients (COMMUNITY, 2024). Embora o protocolo gRPC ofereça vantagens como comunicação binária e maior eficiência, ele não

é suportado nativamente por navegadores, tornando-o menos viável para aplicações web. Assim, a escolha do HTTP visa facilitar o desenvolvimento de clientes e integração com sistemas diversos, enquanto o gRPC é utilizado internamente, na comunicação entre nós, para aproveitar sua baixa latência e maior eficiência.

Além disso, o projeto introduz uma **camada de balanceamento** que utiliza HTTP para redirecionar requisições, aproveitando a necessidade dessa camada para simplificar o uso do sistema. Essa estrutura permite que o balanceador direcione operações de escrita ao nó líder e distribua operações de leitura entre os nós disponíveis, utilizando um algoritmo *round-robin*.

O sistema é organizado em camadas que buscam reduzir o acoplamento e a duplicação de código. As camadas principais incluem:

- **Camada de Interface da Aplicação:** expõe uma API HTTP REST para que usuários e sistemas possam interagir com o banco de dados, realizando operações de leitura, escrita e deleção.
- **Camada de Balanceamento:** gerencia o encaminhamento das requisições, garantindo que operações de escrita sejam tratadas pelo líder, enquanto operações de leitura são distribuídas entre os nós.
- **Camada de Consenso:** utiliza o protocolo Raft para coordenar a consistência dos dados em todos os nós do sistema.
- **Camada de Armazenamento:** realiza operações de leitura e escrita no disco, garantindo confiabilidade e desempenho.

Nas subseções seguintes, cada camada será detalhada com maior profundidade, exemplificando o fluxo de requisições em cenários com um ou múltiplos nós.

2.4.1 Descrição da arquitetura com base em suas camadas

Internamente, a arquitetura do sistema deve buscar reduzir a duplicação e acoplamento do código, para que o processo de desenvolvimento e manutenção do sistema seja facilitado. Por isso, buscou-se a separação de diferentes responsabilidades em camadas bem definidas e hierarquizadas, sendo elas:

1. **Camada de interface da aplicação:** será responsável por promover uma interface para que usuários possam interagir com o sistema, através de operações de leitura, escrita e deleção de dados. Esta interface deverá ser capaz de receber conexões através da internet, ter confiabilidade das entregas, capacidade para lidar com diversos formatos de dados (inclusive binários), ser capaz de lidar com grandes

volumes de dados e ter uma boa usabilidade para o usuário final. Por conta da simplicidade e ampla adoção, será disponibilizado uma API REST.

2. **Camada de Balanceamento:** Esta camada é responsável por definir tqual nó do sistema deve ser responsável por receber cada requisição.
3. **Camada de Consenso:** terá como responsabilidade a comunicação entre os diferentes nós da aplicação em busca do consenso para garantir a consistência dos dados, utilizando mecanismos de votação. Esta camada possui acoplamento com a camada de armazenamento e ambas disponibilizam uma mesma API para as camadas superiores. Para a comunicação entre nós, além de garantir os requisitos anteriores, também é importante levar em consideração a necessidade de baixa latência, evitando que mais nós adicionados ao sistema causem um tempo de resposta muito maior.
4. **Camada de Armazenamento:** Esta camada será orquestrada pela camada de consenso e deverá ser responsável por realizar as operações de leitura e escrita no disco, garantindo confiabilidade e performance da escrita.

Na figura 3 é possível visualizar como o fluxo de uma requisição irá ocorrer através sistema, considerando a operação de apenas um único nó. Há uma API disponibilizada aos usuários finais que encaminha e entende quais comunicações devem ser feitas com o balanceador e este, por fim, transmite as informações para a camada de consenso e armazenamento.

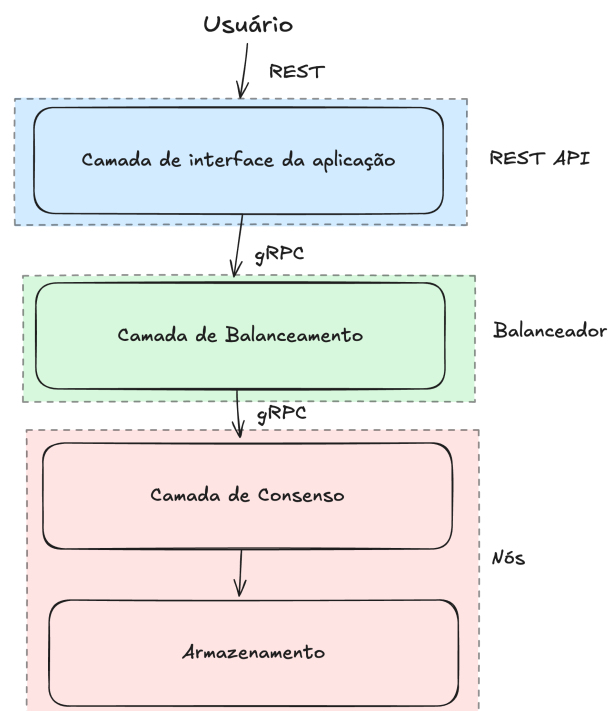


Figura 3 – Requisição considerando um nó único.

Entretanto, conforme o sistema é escalado horizontalmente, a camada de consenso pode precisar se comunicar com outras instâncias da aplicação. Na figura 4, é possível observar que a camada de consenso se comunica com uma API interna própria que permitirá outras ações sob o sistema. Assim é possível separar operações que podem ser executadas por usuários comuns de operações internas.

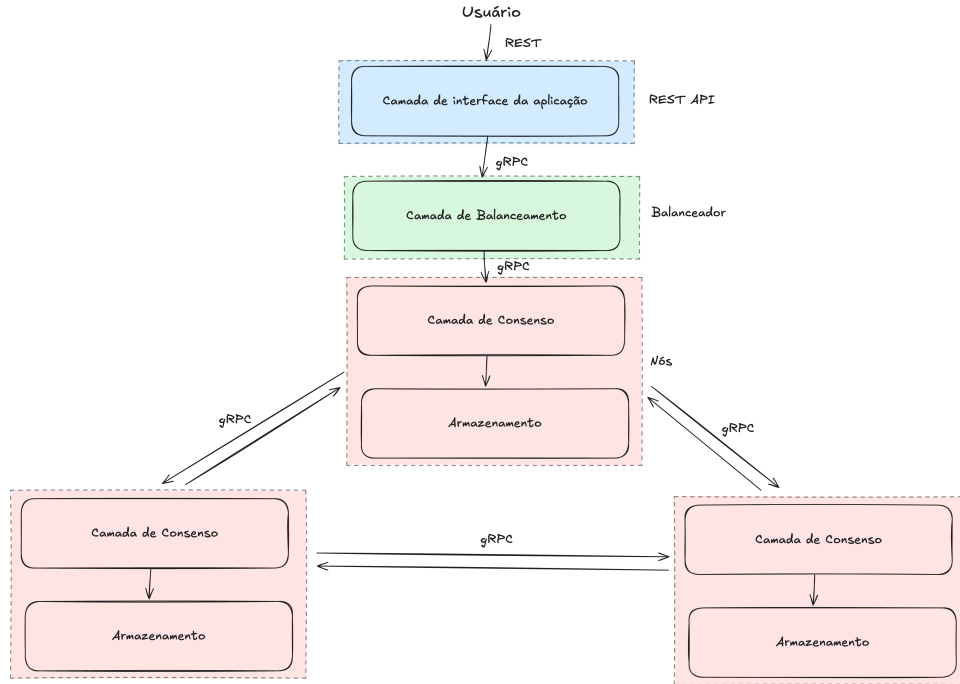


Figura 4 – Requisição considerando três nós que precisam chegar a um consenso.

2.4.2 Descrição das aplicações do sistema

Além de processos que executam os nós do sistema, existe a necessidade de execução de outras tarefas, como balanceamento de carga e interface simplificada para o usuário.

Para separar as responsabilidades na arquitetura, foi escolhido separar o sistema em aplicações diferentes, descritas a seguir.

- **Orquestrador:** Aplicação executada em forma de job durante a inicialização do sistema. Ela é responsável por ler um arquivo de configuração e inicializar todas as outras aplicações, configurando-as corretamente.
- **API REST:** Esta aplicação é responsável por receber requisições através de uma interface REST, recebendo requisições através de rotas HTTP, com corpo da requisição formatado em JSON. Ela pode ser escalada horizontalmente para lidar com um fluxo maior de requisições.
- **Balanceador:** Aplicação responsável por balancear a carga entre todos os nós. Ela disponibiliza uma API em gRPC e balancea a carga para o líder quando necessário,

ou para outros nós utilizando o algoritmo de round-robin. Ela também é responsável por requisições complexas, como adição de nós no sistema, informação que precisa ser propagada para todos os nós do sistema. Também pode ser escalada horizontalmente para lidar com um fluxo maior de requisições.

- **Nós:** Nós do sistema, que executam a camada de consenso e de armazenamento de dados. Estes devem ser sempre escalados horizontalmente, com um número ímpar de nós para que haja sempre um vencedor das eleições.

2.4.3 Descrição dos componentes da arquitetura

O sistema foi escrito em um monolito, compartilhando a mesma base de código entre todas as aplicações. No entanto, há diferentes módulos para cada uma delas e pouco compartilhamento de código.

Por se tratar de uma base de código relativamente pequena, todas as aplicações são compiladas em um único arquivo binário, que é executado usando uma CLI (command line interface), permitindo escolher qual aplicação será executada.

Foram definidos os seguintes componentes:

- **bin:** diretório em que se localizam arquivos binários.
- **cmd:** diretório que contém um subdiretório para cada aplicação e nestes se localizam os códigos *"main"*, responsáveis pela inicialização da aplicação desejada.
- **data:** diretório usado para persistir os arquivos que contém os dados do banco de dados.
- **dto:** diretório compartilhado entre as aplicações contendo os DTOs (Data Transfer Object). Estes são utilizados para traduzir entre diferentes formatos de dados usados através da aplicação.
- **handlers:** diretório onde localiza-se o código responsável por executar as requisições da API Rest. Há um handler para cada grupo de requisições (exemplo: database, responsável pelas operações de banco de dados e admin para as operações de administração do sistema)
- **internal:** diretório compartilhado entre as aplicações que contém código para configurar o sistema (exemplo: ler linha de comando e arquivos de configuração). Também contém código de funções utilitárias, como manipulação de datas.
- **pkg:** diretório compartilhado, usado para código que realiza integração com outras bibliotecas, como cliente gRPC e logger.

- **proto**: diretório que contém as definições de Protocol Buffers (*.proto) para os serviços do sistema, além dos arquivos Go gerados a partir dessas definições. O diretório proto é responsável por definir as interfaces e mensagens utilizadas para comunicação via gRPC entre os diferentes serviços da aplicação, garantindo uma comunicação estruturada, eficiente e consistente.
- **raft**: diretório responsável pelo algoritmo de consenso Raft, contendo a lógica de eleição, replicação de logs e gerenciamento de estados de cada nó (Follower, Candidate, Leader). Fornece métodos para persistência e recuperação de dados críticos, garantindo consistência e tolerância a falhas no cluster distribuído.
- **server**: diretório responsável pelos serviços gRPC e inicialização dos nós. Combina a implementação do Raft com a exposição de métodos, como **Heartbeat**, **RequestVote** e **Set**, permitindo comunicação entre os nós e interações com clientes externos. Também gerencia a configuração, sincronização de acesso e o ciclo de vida do servidor.
- **storage**: diretório responsável pela persistência de dados utilizando a biblioteca Pebble. Implementa operações básicas de armazenamento, recuperação e exclusão de dados com segurança em concorrência através de mutexes. Inclui funcionalidades para inicialização do armazenamento, desligamento seguro e integração com o sistema de logging para monitoramento e diagnóstico.
- **structs**: diretório que define as estruturas de dados utilizadas no sistema, como **LogEntry** e **LogCommand**. Essas estruturas são essenciais para representar as entradas de log do algoritmo *Raft*, encapsulando informações sobre o termo, índice e comando associado, bem como operações e valores relacionados ao banco de dados chave-valor.
- **tools**: contém dois scripts em Python projetados para realizar requisições em lote para criação e leitura dos dados. Esses scripts validam a consistência dos dados retornados pelo serviço de armazenamento e fornecem informações detalhadas sobre os limites de requisições do sistema, contribuindo para testes de desempenho e robustez do serviço.

A Figura 5 apresenta uma visão geral da arquitetura do sistema, destacando cada camada e a forma como elas se relacionam. Observa-se que a camada de Interface da Aplicação (API REST) depende do diretório de servidores e injeta o algoritmo de raft como dependência e este, por sua vez depende do **storage** e das **structs**. A camada de balanceamento depende do servidor e também de clientes gRPC, que decide para qual nó os pedidos devem ir. Na camada de consenso, existe a dependência de um servidor (dessa vez REST ao invés de gRPC) e este depende de um **handler** e este, por sua vez, depende do cliente gRPC. Os diretórios representados na horizontal são compartilhados por toda a

aplicação e possuem poucas dependências entre si, em busca de ter uma arquitetura de simples manutenção.

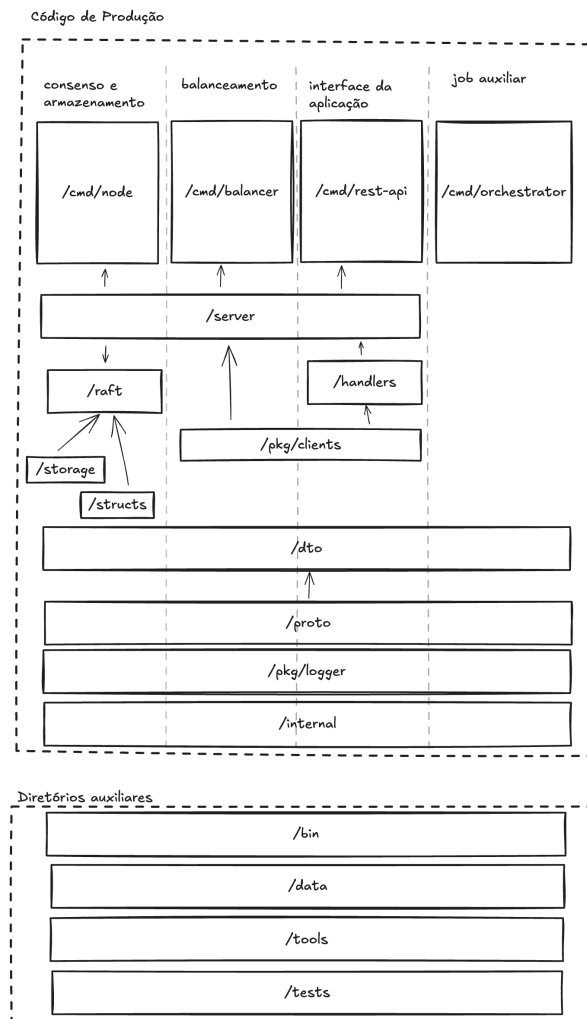


Figura 5 – Visão geral dos componentes e suas interdependências.

2.5 Observabilidade

Conforme sistemas distribuídos escalam e se tornam mais complexos, entendê-los e mantê-los se torna mais difícil para as equipes de desenvolvimento e infraestrutura. Para solucionar este problema, uma solução adotada é a observabilidade para garantir a habilidade para monitorar os sistemas em tempo real.

A observabilidade é um conjunto de práticas que permitem entender o estado interno de um sistema através da coleta e análise de saídas. A coleta de dados pode ser dividida em três pilares: logs, métricas e traces.

- **Logs:** arquivos de texto que registram eventos e atividades de um sistema, permitindo a análise detalhada do que aconteceu com o sistema. Um log pode ser usado para

informar que um erro aconteceu e, com essa informação, descobrir quando o erro ocorreu, o que estava sendo executado e quais as suas consequências.

- **Traces:** dados sobre como uma requisição atravessa o sistema e seus nós. Estes dados contêm informações sobre a latência do sistema e quais partes estão sendo mais ofensoras no tempo de resposta. Através do uso de tracing, é possível encontrar problemas na execução e gargalos para a performance da aplicação.

Através das coletas e análises destes dados, a análise de problemas se tornará mais fácil, diminuindo o tempo para correção de erros e permitindo melhorias de performance com maior facilidade. Além disso, também facilitará o processo de testagem manual da aplicação em situações de alta carga.

2.6 Banco de Dados Chave-Valor

Um banco de dados chave-valor é um tipo de banco de dados não relacional que utiliza um modelo simples de pares *chave-valor* para armazenar dados. Neste modelo, cada dado armazenado é associado a uma chave única, permitindo acesso rápido e eficiente aos valores correspondentes. Esse tipo de banco de dados é caracterizado por sua simplicidade, desempenho e capacidade de escalabilidade horizontal, tornando-o ideal para aplicações que exigem alto desempenho e gerenciamento de grandes volumes de dados ([Amazon Web Services, 2024](#); [NAPOLEON, 2024](#)).

Os sistemas gerenciadores de bancos de dados chave-valor são utilizados em diferentes cenários, especialmente quando não há necessidade de dados estruturados, como no modelo relacional. Como exemplo, para salvar um arquivo, em um sistema de armazenamento, o modelo chave-valor, utilizando o caminho do arquivo como chave, pode ser mais adequado do que modelo relacional. Outro aspecto importante é uma latência menor, já que modelos mais complexos, como o relacional, adicionam tempo de processamento para conseguir traduzir as consultas na linguagem adotada para operações no sistema.

2.6.1 Aplicações que Utilizam Bancos de Dados Chave-Valor

O modelo de banco de dados chave-valor destaca-se por sua simplicidade e eficiência no acesso a dados, geralmente proporcionando baixa latência. Graças a essas características, ele se mostra adequado para uma ampla variedade de cenários em que o desempenho e a disponibilidade são cruciais. Abaixo, apresentam-se alguns exemplos de aplicações práticas que fazem uso de bancos de dados chave-valor:

- **Redes Sociais:** Plataformas como o Twitter utilizam bancos de dados chave-valor, como Redis, para armazenar sessões de usuários, contagens de seguidores e outras

informações que requerem acesso rápido ([CARLSON, 2013](#)).

- **E-commerce:** Sites de comércio eletrônico utilizam esse modelo para gerenciar carrinhos de compras, preferências de usuários e histórico de navegação, proporcionando respostas rápidas e melhor experiência ao cliente ([LAKSHMAN; MALIK, 2010](#)).
- **Sistemas de Cache:** O Redis, por exemplo, é frequentemente empregado como cache em aplicações web para acelerar o acesso a dados consultados com frequência ([IMASTERS, 2024](#)).

3 Método do trabalho

O desenvolvimento deste trabalho está dividido em cinco fases: estudo do tema, especificação dos requisitos, planejamento do projeto, implementação e testes.

3.1 Estudo do tema

A primeira etapa, estudo do tema, envolve o estudo de aspectos teóricos necessários para entender os possíveis e diferentes sistemas que poderiam ser criados dentro do tema.

3.2 Especificação de Requisitos

Na especificação dos requisitos, o objetivo é detalhar os requisitos do sistema, separando em duas partes: as funcionalidades que o sistema deve implementar (requisitos funcionais) e outros aspectos não funcionais aos quais o sistema deve atender (requisitos não funcionais).

3.3 Planejamento do Projeto

Na etapa de planejamento do projeto é definida a arquitetura (separando a responsabilidade dos diferentes componentes e definindo como eles se comunicam) que o sistema deve utilizar para conseguir atender aos requisitos previamente especificados. Nesta etapa as tecnologias utilizadas também são definidas para alcançar os objetivos e idealizar como a aplicação deverá ser testada para validar que os requisitos foram atendidos.

3.4 Implementação

A fase de implementação consiste na construção e desenvolvimento dos componentes do sistema de acordo com a arquitetura previamente planejada.

Os módulos do sistema são implementados de forma incremental e iterativa, permitindo a integração progressiva das funcionalidades. Para cada componente desenvolvido, são realizados testes com o objetivo de validar o funcionamento de partes específicas do sistema. Isso ajuda a identificar e corrigir bugs no estágio inicial, reduzindo o impacto de erros no desenvolvimento .

Durante o processo de desenvolvimento, o projeto utilizou métodos de ciclos curtos de trabalho e entrega incremental de funcionalidades. O uso de controle de versão, por

meio da ferramenta Git também é fundamental para gerenciar alterações no código e facilitar o trabalho em equipe.

3.5 Testes

Por fim, na fase de testes são realizados testes sobre o sistema todo garantindo seu funcionamento, integração entre os componentes e entendendo se as características especificadas foram atendidas.

4 Especificação de Requisitos

O sistema proposto é um *Gerenciador de Banco de Dados Chave-Valor Distribuído*, cujo propósito é lidar com grandes volumes de dados e garantir alta disponibilidade para operações de leitura (**GET**). Nesse contexto, adotam-se conceitos de sistemas distribuídos e algoritmos de consenso — em particular, o *Raft* ([ONGARO; OUSTERHOUT, 2014](#)) — para equilibrar escalabilidade e consistência.

Dada a natureza distribuída, o sistema é concebido para operar em múltiplas instâncias ou contêineres que se comunicam entre si, evitando pontos únicos de falha e permitindo crescimento horizontal da quantidade de nós. Tal abordagem segue a linha de sistemas como o Cassandra ([LAKSHMAN; MALIK, 2010](#)), focados em expansão dinâmica para suportar alta demanda.

A seguir, detalham-se os requisitos funcionais (capacidade de armazenar, recuperar, atualizar e remover dados) e os requisitos não funcionais (escalabilidade horizontal, consistência, alta performance, tolerância a falhas e observabilidade). Cada requisito está alinhado às decisões de projeto e às tecnologias adotadas.

4.1 Requisitos Funcionais

4.1.1 RF1 – Salvar Dados Binários

O sistema deve permitir que o usuário (ou outra aplicação) envie um valor textual associado a uma chave textual. Na prática:

- A requisição (*chave, valor*) chega ao nó líder, que persiste localmente e replica a informação via *log* de consenso (*Raft*).

4.1.2 RF2 – Ler Dados com Base em uma Chave

O sistema deve retornar o valor previamente armazenado, dada a chave solicitada. Essa funcionalidade:

- É essencial em cenários de leitura intensiva, pois muitos sistemas chave-valor servem como cache ou depósito de consultas rápidas.
- Pode ser atendida por qualquer nó do cluster (dependendo de como o balanceador redireciona as requisições).

4.1.3 RF3 – Deletar Dados com Base em uma Chave

O sistema deve remover o valor correspondente a uma chave específica:

- Uma requisição de deleção (DELETE, em API REST) retira o registro da camada de armazenamento.

4.1.4 RF4 – Atualizar Dados com Base em uma Chave

O sistema deve permitir a sobrescrita de um valor já existente:

- A operação pode ser feita via PUT da API REST, substituindo completamente o valor anterior.

4.2 Requisitos Não Funcionais

4.2.1 RNF1 – Escalabilidade Horizontal

O sistema deve suportar o acréscimo de novas instâncias para acompanhar o crescimento da demanda, em especial para leituras:

- Foi planejado para executar em vários contêineres, podendo escalar estaticamente.
- Um balanceador de carga interno direciona as requisições entre os nós, otimizando o uso dos recursos.

4.2.2 RNF2 – Consistência dos Dados

O sistema deve manter seus dados sincronizados entre todos os nós:

- O algoritmo Raft ([ONGARO; OUSTERHOUT, 2014](#)) foi escolhido em detrimento de outras opções (como Paxos ([LAMPORT, 2005](#))) devido à sua clareza de implementação.
- Toda operação de escrita (inserção, remoção, atualização) é registrada no *log* do líder e replicada para os demais nós, mantendo a consistência forte.

4.2.3 RNF3 – Alta Performance

O desempenho deve ser adequado para um sistema distribuído, sobretudo em operações de leitura:

- O uso de Go (goroutines, gRPC) busca reduzir latências e facilitar concorrência.

- Observou-se, no projeto, que picos de inserção precisam de cuidados especiais, pois o líder centraliza as escritas.

4.2.4 RNF4 – Tolerância a Falhas

O sistema deve continuar operacional frente à queda de um ou mais nós:

- O protocolo Raft e a replicação de logs permitem eleição de um novo líder, quando o atual falha.

4.2.5 RNF5 – Observabilidade

A solução deve oferecer meios para monitorar e diagnosticar o sistema:

- Logs estruturados e *tracing* facilitam a detecção de anomalias.

4.3 Considerações Finais

Em suma, as funcionalidades previstas (armazenamento, leitura, atualização e deleção de dados binários) aliadas aos requisitos de escalabilidade, tolerância a falhas e alta consistência definem o escopo do presente sistema de banco de dados chave-valor distribuído. A adoção do algoritmo Raft, a implementação de um balanceador de carga e o suporte para múltiplos contêineres refletem a busca por um ambiente *clusterizado* e resiliente, atendendo às melhores práticas em arquiteturas de sistemas distribuídos (LAKSHMAN; MALIK, 2010; ONGARO; OUSTERHOUT, 2014).

O detalhamento desses requisitos guiou todo o processo de planejamento, implementação e testes do projeto, assegurando alinhamento às necessidades de aplicações que demandam grande volume de leituras e integridade robusta dos dados.

5 Desenvolvimento do Trabalho

5.1 Tecnologias Utilizadas

O desenvolvimento deste trabalho envolveu a utilização de uma série de tecnologias e ferramentas escolhidas pela sua adequação aos requisitos do sistema, com foco em consistência forte, escalabilidade horizontal e tolerância a falhas. A seguir, apresentamos as principais tecnologias utilizadas e as razões que motivaram sua escolha.

5.1.1 Linguagem de Programação Go

A linguagem Go (*Golang*) foi escolhida como base para o desenvolvimento do sistema devido às seguintes características:

- **Alto Desempenho:** Go é uma linguagem compilada que gera código de máquina otimizado, garantindo baixa latência e alto desempenho, aspectos cruciais para sistemas distribuídos.
- **Paralelismo Simplificado:** A implementação de goroutines e canais permite a criação de aplicações altamente concorrentes de forma simples, ideal para gerenciar múltiplos nós de um cluster.
- **Bibliotecas Robustas:** Go oferece suporte nativo para criação de servidores HTTP e gRPC, além de pacotes eficientes para manipulação de arquivos, sincronização de threads e comunicação em rede.
- **Facilidade de Manutenção:** A sintaxe simples e legível de Go, aliada ao gerenciamento de dependências com `go modules`, reduz a complexidade no desenvolvimento e manutenção do código.

5.1.2 Protocol Buffers (Protobuf)

Protocol Buffers, ou Protobuf, é uma tecnologia de serialização de dados utilizada para definir os serviços e mensagens do sistema. Os motivos para sua adoção incluem:

- **Eficiência:** As mensagens geradas são compactas, reduzindo o consumo de banda e melhorando a performance em redes distribuídas.
- **Interoperabilidade:** O Protobuf permite que os serviços sejam definidos de forma neutra em relação à linguagem de programação, facilitando futuras integrações.

- **Automação:** A geração automática de código simplifica a implementação de servidores e clientes gRPC, garantindo consistência entre os nós do sistema.

5.1.3 gRPC

O protocolo gRPC foi utilizado como base para a comunicação entre os nós do cluster. Ele foi escolhido pelas seguintes razões:

- **Comunicação Bidirecional:** Suporte nativo a chamadas *streaming*, permitindo comunicação contínua e eficiente entre líder e seguidores.
- **Interoperabilidade:** Compatível com diversas linguagens de programação, o que pode facilitar a extensão do sistema no futuro.
- **Facilidade de Integração:** Integra-se nativamente com Protobuf, acelerando o desenvolvimento e garantindo consistência na troca de mensagens.

5.1.4 Gerenciamento de Configuração com YAML

O formato YAML foi utilizado para configurar os nós do cluster em implementações dinâmicas. As principais vantagens de sua utilização incluem:

- **Leitura Humana:** A sintaxe simples do YAML facilita a configuração e a compreensão das definições do cluster.
- **Flexibilidade:** Permite a definição de parâmetros, como portas e identificadores dos nós, sem a necessidade de recompilar o código.

5.1.5 Gerenciamento de Estado e Persistência

Para garantir tolerância a falhas, o sistema armazena informações críticas em disco utilizando as bibliotecas nativas de Go para manipulação de arquivos e serialização (`encoding/gob`). Essa abordagem foi escolhida devido à sua simplicidade e eficiência, permitindo que os nós recuperem seu estado rapidamente após falhas ou reinicializações.

5.1.6 Controle de Concorrência com `sync` e Goroutines

A biblioteca padrão de sincronização (`sync`) foi usada para implementar mecanismos de controle de concorrência, como mutexes e `sync.WaitGroup`. Além disso, o uso extensivo de goroutines permitiu que o sistema realizasse operações de forma paralela, aumentando o desempenho e a escalabilidade.

5.1.7 Logger para Monitoramento

Para a produção de logs do sistema, foi desenvolvido um pacote próprio localizado na pasta `pkg/logger`. Este pacote funciona como um *wrapper* para a biblioteca **Zap**, uma ferramenta de logs estruturados desenvolvida pela Uber Technologies, Inc. [Uber Technologies, Inc. \(2024\)](#).

A utilização da biblioteca Zap trouxe diversas vantagens:

- **Desempenho Elevado:** Otimizada para aplicações de alta performance, garantindo eficiência mesmo em sistemas com grande volume de dados.
- **Logs Estruturados:** Permite o registro de informações em formatos como JSON, facilitando a análise e o processamento automatizado.
- **Configuração Flexível:** Suporta diferentes níveis de log (INFO, DEBUG, ERROR) e formatos de saída, adaptando-se a variados cenários operacionais.

O pacote `pkg/logger` padroniza a geração de logs em todo o sistema, evitando repetições de código e garantindo consistência. Isso proporciona:

1. **Consistência:** Mantém o mesmo formato e estrutura de logs em todas as partes do sistema.
2. **Monitoramento Eficiente:** Facilita a identificação de falhas e o rastreamento de problemas em sistemas distribuídos.
3. **Manutenção Simplificada:** Permite ajustes na lógica de logs diretamente no pacote, refletindo-se em toda a aplicação.

Além disso, o logger foi configurado para registrar informações críticas, como mensagens de erro, estados internos dos nós do cluster e dados de depuração, contribuindo significativamente para a observabilidade e a manutenção do sistema distribuído.

5.1.8 Outras Ferramentas

- **Git e GitHub:** Utilizados para versionamento e colaboração no desenvolvimento.
- **Makefiles:** Automatizaram a execução de comandos comuns, como geração de código Protobuf e compilação do projeto.

5.2 Aplicações Desenvolvidas

Por se tratar de um banco de dados, construído para ser utilizado principalmente por outros sistemas de software, o projeto foi desenvolvido com um grande enfoque em recursos técnicos e lógicos, como o algoritmo de replicação e armazenamento.

É um sistema com poucos recursos para o usuário final, como interface gráfica ou protocolos fáceis de serem utilizados. O protocolo gRPC, por exemplo, apesar de ser bem comum para comunicação entre sistemas distribuídos, não é tão adotado quanto APIs REST e possui uma curva de aprendizado maior.

O sistema foi categorizado em partes que possuem dependência entre si. A primeira delas, nomeada de **Aplicações Core**, contém o que usualmente chama-se de back-end, isto é, que não são comumente utilizadas por um usuário final, mas por outras aplicações.

Além disso, também foi construído um código de *front-end* simples, visando principalmente o auxílio na condução de testes para mostrar o funcionamento do sistema como um todo.

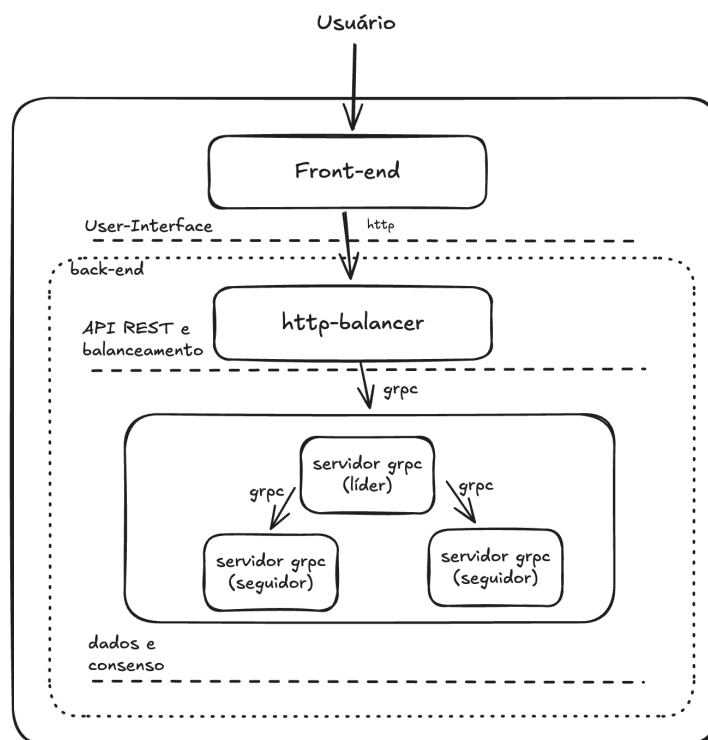


Figura 6 – Fluxo das requisições no sistema.

5.2.1 CLI - Aplicações Core

A primeira parte do sistema é a *CLI* (*Command-Line Interface*). Ela é composta por três aplicações contidas em um mesmo arquivo binário e que compartilham a mesma base de código.

Foi utilizada uma estrutura de comandos que permite facilmente adicionar novas aplicações, se necessário. Ela é compilada através de um arquivo `Makefile` e foi criada de forma a produzir instruções para o usuário, conforme o exemplo a seguir.

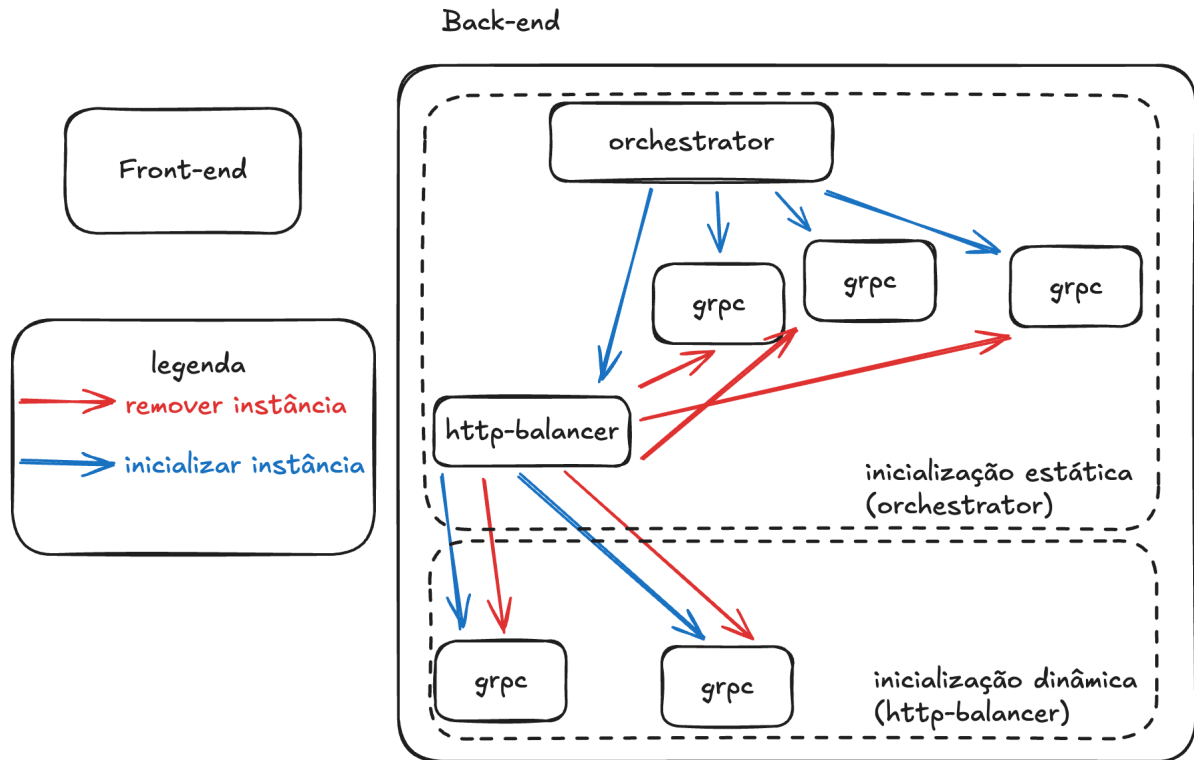


Figura 7 – Exemplo de inicialização com três instâncias iniciais e duas dinâmicas.

```
$/bin/cli help
```

```
Single binary with multiple commands
```

```
Usage:
```

```
cli [command]
```

```
Available Commands:
```

```
completion    Generate the autocompletion script for the specified shell
grpc           Start grpc server
help           Help about any command
http-balancer  Start rest http api
orchestrator   Orchestrates raft nodes
```

```
Flags:
```

```
-h, --help    help for cli
```

Use `"cli [command] --help"` for more information about a command.

Neste resultado do console ao executar o comando de ajuda da CLI, é possível ver a presença de um comando para ajuda, um gerador de scripts de *autocomplete* (para terminal *bash* e *zsh*) e outras três aplicações: (i) `grpc` (Orquestrador da Aplicação) - que cria servidores gRPC; (ii) `http-balancer` (API REST e Balanceador de Carga) - que cria um servidor REST HTTP que irá balancear a carga dos sistemas; e (iii) `orchestrator` (Orquestrador da Aplicação) - uma aplicação que é responsável por coordenar as outras.

5.2.1.1 Orquestrador da Aplicação

O orquestrador da aplicação é configurado através de um arquivo `yaml` - tipo de arquivo amplamente usado em configurações de aplicações, como o Docker. Ele é o componente principal das Aplicações Core e, além de carregar as configurações, também inicializa as instâncias das outras aplicações, como os servidores gRPC e o balanceador de carga.

5.2.1.2 API REST e Balanceador de Carga

Esta aplicação possui duas responsabilidades: (i) produzir uma interface mais simples para usuários finais - uma API REST, que é mais fácil de ser utilizada; e (ii) balancear a carga entre os diferentes servidores gRPC.

O balanceador de carga é um intermediador responsável por realizar requisições às instâncias finais do banco de dados, de forma a distribuir a carga igualmente entre as mesmas.

A API REST realiza uma simplificação das interfaces de `protobuf` através do uso de APIs Rest. Ela é organizada em dois grupos de rotas: *db* e *admin*. A primeira é responsável por executar os comandos de salvar, ler e deletar dados do banco de dados. Para isso, ela realiza requisições através da camada de balanceamento.

Além de ações sobre o armazenamento do banco de dados, esta também é responsável por adicionar ou remover instâncias (inicialização dinâmica de instâncias) do banco de dados para além da inicialização estática.

5.2.1.3 Servidores gRPC

Por fim, o sistema também possui uma aplicação responsável por realizar as manipulações nos dados (leitura, gravação e deleção de dados).

Estes se comunicam entre-si exclusivamente através do gRPC e também possuem uma interface utilizando o mesmo protocolo. Estas aplicações também são responsáveis por garantir o consenso entre si.

5.2.2 Interface Web e Integração com o Sistema

Para testar e validar, de forma prática, o funcionamento do sistema de banco de dados distribuído, foi desenvolvido um front-end que permite inserir, consultar e recuperar arquivos.

Além da implementação do sistema gerenciador distribuído, esse front-end contribui com os testes e evidencia, de maneira clara, o uso do modelo chave-valor. Construído com o *framework* Angular (TypeScript), ele ilustra como um usuário pode interagir facilmente com o sistema, inserindo, consultando e recuperando arquivos PDF a partir da base de dados distribuída.

5.2.2.1 Principais Funcionalidades da Interface Web

- **Interface de Upload de Arquivo:** Uma página em Angular exibe um campo para seleção de um arquivo PDF e outro para a inserção de um nome (chave) associado ao documento. Após o usuário escolher o arquivo, o front-end lê o conteúdo binário e o converte em Base64, garantindo uma transmissão padronizada via HTTP.
- **Conversão para Base64:** O componente TypeScript responsável pelo upload utiliza a `FileReader` API do navegador para ler o PDF e convertê-lo em uma string Base64. Essa conversão torna o PDF adequado para ser armazenado no banco de dados chave-valor, que exige dados textuais para chave e valor.
- **Integração com a API REST do Banco de Dados:** Após a conversão, o usuário insere o nome desejado para o arquivo. Ao clicar em "Registrar", o front-end realiza uma requisição HTTP POST à API do banco de dados distribuído, enviando a chave (nome do arquivo) e o valor (string Base64 do PDF).
- **Armazenamento no Banco de Dados Distribuído:** A API registra a chave e o valor no cluster distribuído, garantindo consistência, disponibilidade e escalabilidade. Assim, o PDF fica armazenado de forma segura e tolerante a falhas, podendo ser recuperado futuramente.

5.2.2.2 Recuperação de Arquivos PDF

Como parte da validação do sistema distribuído, foi implementada a funcionalidade de recuperar arquivos já armazenados, destacando a confiabilidade e a facilidade de acesso aos dados.

Além do registro de novos PDFs, o front-end implementa um módulo em TypeScript para recuperar arquivos já armazenados. O usuário fornece a chave (nome do PDF), e o componente realiza uma requisição HTTP GET para a API do banco de dados, obtendo a

string Base64 correspondente. A seguir, esse valor é decodificado e convertido novamente em PDF, permitindo que o usuário baixe o documento.

Esse processo foi escolhido por sua simplicidade e eficiência, além de evidenciar que o requisito funcional de armazenamento e recuperação de dados do sistema foi implementado corretamente.

O funcionamento é mostrado no trecho de código a seguir, que apresenta a função `generatePDF()`:

```
generatePDF(): void {
  if (this.inputBase64) {
    try {
      // Remove o prefixo "data:application/pdf;base64," se existir
      const base64Data = this.inputBase64.startsWith('data:application/pdf;base64,')
        ? this.inputBase64.split(',')[1]
        : this.inputBase64;

      // Decodifica a string Base64 em binário
      const byteCharacters = atob(base64Data);
      const byteNumbers = Array.from(byteCharacters).map(char => char.charCodeAt(0));
      const byteArray = new Uint8Array(byteNumbers);
      const blob = new Blob([byteArray], { type: 'application/pdf' });

      // Cria um link para download do PDF
      const url = window.URL.createObjectURL(blob);
      const a = document.createElement('a');
      a.href = url;
      a.download = 'generated.pdf';
      document.body.appendChild(a);
      const event = new MouseEvent('click', { bubbles: true, cancelable: true });
      a.dispatchEvent(event);

      document.body.removeChild(a);
      setTimeout(() => {
        window.URL.revokeObjectURL(url);
      }, 1000);
      window.open(url, '_blank');
    } catch (e) {
      this.error = 'Erro ao gerar o PDF. Verifique a chave inserida.';
    }
  }
}
```

```
    } else {  
        this.error = 'Por favor, insira uma chave válida.';  
    }  
}
```

5.2.2.3 Funcionamento Detalhado

- **Requisição HTTP:** O front-end solicita ao banco de dados a string Base64 associada à chave fornecida.
- **Decodificação Base64:** A string é convertida em binário por meio da função `atob()`.
- **Criação do PDF:** Os dados binários são encapsulados em um Blob do tipo PDF, permitindo sua manipulação no navegador.
- **Download/Visualização:** É criado dinamicamente um elemento `a` para acionar o download do arquivo, que também pode ser aberto em uma nova aba.
- **Tratamento de Erros:** Em caso de falhas, o usuário é notificado por meio de mensagens de erro.

Portanto, embora este front-end em Angular (TypeScript) seja apenas um exemplo de utilização, ele mostra a facilidade e a flexibilidade de integrar sistemas distribuídos de armazenamento chave-valor a aplicações web modernas, evidenciando o potencial da solução proposta neste trabalho.

5.3 Implementação e Algoritmos

Este trabalho propõe o desenvolvimento de um sistema distribuído para um banco de dados chave-valor, com foco em consistência forte, escalabilidade horizontal e tolerância a falhas. Para atingir esses objetivos, foi projetado um conjunto de componentes interconectados, integrando o algoritmo de consenso Raft com serviços de comunicação eficientes e persistência de dados. O Raft foi escolhido como o núcleo do sistema, pois oferece simplicidade e robustez no gerenciamento de replicação de logs e na coordenação entre os nós do cluster distribuído.

Além do algoritmo Raft, o sistema utiliza gRPC para comunicação entre os nós e Protocol Buffers (Protobuf) para definição e serialização das mensagens. A arquitetura modular e flexível do sistema permite a inicialização dinâmica do cluster, garantindo adaptabilidade em diferentes cenários. Nesta seção, apresentamos a arquitetura do sistema, detalhamos o funcionamento do Raft, sua integração com os serviços de comunicação e as decisões técnicas que sustentam o desenvolvimento do projeto.

5.3.1 Implementação do Algoritmo Raft

Para garantir a consistência e a tolerância a falhas no banco de dados distribuído, foi implementado o algoritmo de consenso *Raft*. O *Raft* é amplamente utilizado em sistemas distribuídos por sua simplicidade e clareza em comparação com outros algoritmos de consenso, como o Paxos. A escolha pelo *Raft* se alinha aos objetivos do projeto de fornecer um sistema robusto, escalável e confiável.

5.3.1.1 Estrutura Geral do Nó Raft

Cada nó do cluster não apenas implementa uma instância do algoritmo *Raft*, mas também incorpora um servidor gRPC que expõe os serviços necessários para a comunicação entre os nós e com clientes externos. A estrutura principal do nó é definida pela combinação da estrutura **Raft** e do **Server**, integrando os componentes de comunicação e lógica de consenso.

A estrutura **Server** é responsável por:

- **Serviços gRPC:** Implementa os métodos definidos nos arquivos `.proto`, como `Heartbeat`, `RequestVote`, `Set`, `Delete` e `Get`.
- **Interação com o Raft:** Cada método gRPC delega as operações à instância **Raft** correspondente, garantindo que a lógica de consenso seja aplicada a todas as operações.
- **Sincronização:** Utiliza mutexes para garantir a segurança em concorrência durante o acesso e modificação do estado interno.

A inicialização do servidor é realizada pela função **Start**, que configura o servidor gRPC, registra os serviços e inicia a escuta na porta especificada. Além disso, o servidor monitora sinais do sistema operacional para realizar um desligamento gracioso, garantindo a persistência do estado e a integridade do sistema.

5.3.2 Implementação do Servidor e Inicialização do Cluster

A implementação do servidor e a inicialização do cluster são componentes essenciais para o funcionamento do sistema distribuído. Eles permitem que múltiplos nós operem em conjunto, comunicando-se via gRPC e coordenando-se através do algoritmo *Raft*.

5.3.2.1 Servidor gRPC Integrado ao Raft

O servidor é implementado pela estrutura **Server**, que combina a funcionalidade do gRPC com a lógica do *Raft*:

```

type Server struct {
    mu      sync.Mutex
    raft    *raft.Raft
    logger  logger.Logger

    grpc *grpc.Server

    pb.UnimplementedRaftServer
    pb.UnimplementedDatabaseServer
}

```

Os principais pontos de integração incluem:

- **Métodos Raft:** Implementação dos métodos `Heartbeat` e `RequestVote`, que são chamados por outros nós do cluster.
- **Operações do Banco de Dados:** Implementação dos métodos `Set`, `Delete` e `Get`, permitindo interações com o armazenamento chave-valor.
- **Comunicação Segura:** Uso de mutexes para sincronizar o acesso ao estado do servidor, prevenindo condições de corrida.

5.3.2.2 Inicialização e Configuração do Cluster

A inicialização do cluster é realizada em duas abordagens distintas, evidenciando a flexibilidade do sistema:

5.3.2.2.1 Inicialização Estática

No primeiro cenário, os nós do cluster são definidos estaticamente no código:

```

nodes := make(map[raft.ID]*raft.Node)
nodes["A"] = raft.NewNode("[:]:8080")
nodes["B"] = raft.NewNode("[:]:8081")
nodes["C"] = raft.NewNode("[:]:8082")

```

Cada nó é iniciado em uma goroutine separada, permitindo a execução concorrente dos servidores:

```

go func() {
    server := server.CreateServer(

```

```

        raft.MakeRaft("A", nodes),
    )
    server.Start(":8080")
}()

```

5.3.2.2.2 Inicialização Dinâmica via Arquivo de Configuração

No segundo cenário, a configuração dos nós é carregada a partir de um arquivo YAML, permitindo maior flexibilidade e escalabilidade:

```

file, err := os.ReadFile("raft.yaml")
var config Config
yaml.Unmarshal(file, &config)

```

Os nós são criados dinamicamente com base nas informações do arquivo:

```

for _, s := range config.RaftCluster.Servers {
    nodes[s.ID] = raft.NewNode(fmt.Sprintf("[::]:%s", s.Port))
}

```

5.3.2.3 Execução e Gerenciamento do Cluster

Em ambos os casos, os servidores são iniciados e gerenciados utilizando `sync.WaitGroup` para sincronizar a execução das goroutines:

```

var wg sync.WaitGroup
for _, s := range config.RaftCluster.Servers {
    wg.Add(1)
    go func(srv Server) {
        defer wg.Done()
        server := server.CreateServer(
            raft.MakeRaft(srv.ID, nodes),
        )
        server.Start(srv.Port)
    }(s)
}
wg.Wait()

```

Essa abordagem assegura que todos os nós do cluster sejam iniciados corretamente e permaneçam em execução durante o funcionamento do sistema.

5.3.2.4 Integração com o Algoritmo Raft e o Sistema

A implementação do servidor e a inicialização do cluster são cruciais para a integração completa do algoritmo Raft ao sistema. Elas permitem:

- **Comunicação entre Nós:** Utilizando gRPC, os nós podem trocar mensagens de forma eficiente e padronizada.
- **Execução Concorrente:** A utilização de goroutines e sincronização permite que o sistema aproveite o paralelismo, melhorando a performance.
- **Flexibilidade de Configuração:** A possibilidade de definir os nós via arquivo de configuração facilita a adaptação do sistema a diferentes ambientes e necessidades.

5.3.2.5 Benefícios para o Projeto

A implementação detalhada do servidor e do processo de inicialização do cluster trouxe os seguintes benefícios ao projeto:

- **Escalabilidade Horizontal:** Facilita a adição ou remoção de nós no cluster sem a necessidade de alterações significativas no código.
- **Robustez e Confiabilidade:** O tratamento adequado de sinais do sistema operacional e a sincronização de acesso garantem uma operação estável.
- **Adaptabilidade:** A configuração dinâmica permite que o sistema seja facilmente adaptado a diferentes cenários de uso e requisitos.

Cada nó do cluster implementa uma instância do algoritmo *Raft*, mantendo estados críticos e participando do processo de consenso. A estrutura principal do nó é definida pela estrutura *Raft*, que contém os seguintes componentes:

- **Identificador do Nó (**me**):** Representa o ID único do nó dentro do cluster.
- **Estados Persistentes:**
 - **currentTerm:** O termo atual do nó, utilizado para identificar a validade de mensagens.
 - **votedFor:** ID do candidato que recebeu o voto do nó no termo atual.
 - **logEntry:** Registro das entradas de log que precisam ser replicadas.
- **Estados Voláteis:**
 - **commitIndex:** Índice da maior entrada de log conhecida como comprometida.

- `lastApplied`: Índice da última entrada de log aplicada ao estado do nó.

- **Estados Voláteis em Líderes:**

- `nextIndex`: Para cada nó, o índice da próxima entrada de log que o líder enviará para replicação.
- `matchIndex`: Para cada nó, o índice da maior entrada de log conhecida replicada naquele nó.

5.3.2.6 Estados do Nó

O algoritmo *Raft* define três estados para os nós: *Follower* (seguidor), *Candidate* (candidato) e *Leader* (líder). A transição entre esses estados é gerenciada pela enumeração `State`, como mostrado abaixo:

```
type State int

const (
    Follower State = iota + 1
    Candidate
    Leader
)
```

5.3.2.7 Persistência de Estado

A persistência de estado é uma característica fundamental para garantir a tolerância a falhas no sistema. Informações críticas, como o termo atual, o nó votado, o índice de commit, e as entradas de log, são armazenadas em disco para permitir que os nós recuperem seu estado após falhas ou reinicializações.

A persistência é gerenciada por funções específicas que serializam o estado do nó em um arquivo local. Esses arquivos são organizados em uma estrutura que identifica cada nó pelo seu ID exclusivo. A serialização é realizada utilizando o pacote `encoding/gob`, que converte as estruturas de dados do Go em um formato adequado para armazenamento.

Por exemplo, a função responsável por salvar o estado coleta todos os dados relevantes do nó em uma estrutura e os serializa para um arquivo específico. Essa abordagem garante que o estado do nó possa ser carregado novamente de maneira confiável, mantendo a integridade do sistema mesmo em cenários de falhas inesperadas.

Essa estratégia contribui significativamente para a robustez do sistema distribuído, reduzindo os impactos de falhas e otimizando a recuperação de nós.

5.3.2.8 Mecanismo de Eleições

Quando um nó não recebe *heartbeats* do líder dentro de um intervalo de tempo aleatório, ele se torna um *Candidate* e inicia uma eleição. O processo de eleição é realizado pelas funções `broadcastRequestVote` e `sendRequestVote`, que solicitam votos dos outros nós:

```
func (r *Raft) broadcastRequestVote() {
    args := dto.VoteArgs{
        Term:      r.currentTerm,
        CandidateID: r.me,
    }

    for i := range r.nodes {
        go func(i ID) {
            r.sendRequestVote(i, args)
        }(i)
    }
}
```

Se o candidato receber votos da maioria dos nós, ele se torna o líder e começa a enviar *heartbeats* para os seguidores.

5.3.2.9 Replicação de Logs e *Heartbeats*

O líder é responsável por manter os seguidores atualizados, replicando as entradas de log através de *heartbeats*. A função `broadcastHeartbeat` envia periodicamente mensagens aos seguidores contendo novas entradas de log e informações de commit:

```
func (r *Raft) broadcastHeartbeat() {
    for i := range r.nodes {
        var args dto.HeartbeatArgs

        args.Term = r.currentTerm
        args.LeaderID = r.me
        args.LeaderCommit = r.commitIndex

        prevLogIndex := max(r.nextIndex[i]-1, 0)
        if r.getLastIndex() > prevLogIndex {
            args.PrevLogIndex = prevLogIndex
            args.PrevLogTerm = r.logEntry[prevLogIndex].Term
        }
    }
}
```

```
        args.Entries = r.logEntry[prevLogIndex:]
    }

    go func(i ID, args dto.HeartbeatArgs) {
        r.sendHeartbeat(i, args)
    }(i, args)
}
```

Os seguidores, ao receberem um *heartbeat*, atualizam seus logs e confirmam o recebimento, permitindo que o líder avance o `commitIndex` quando a maioria dos nós tiver replicado as entradas.

5.3.2.10 Aplicação de Entradas de Log

As entradas de log no algoritmo *Raft* representam comandos que precisam ser aplicados ao estado do sistema, garantindo a consistência entre os nós do cluster. Esses comandos incluem operações básicas de armazenamento de chave-valor, como **SET**, para adicionar ou atualizar um valor associado a uma chave, e **DELETE**, para remover uma chave específica.

A aplicação desses comandos é gerenciada pela função **apply**, que verifica as entradas do log já comprometidas e as executa sequencialmente. Durante essa execução, o sistema realiza as seguintes etapas:

- **Validação do Índice de Commit:** Antes de processar as entradas, a função garante que o índice de commit (`commitIndex`) é maior ou igual ao índice da última entrada aplicada (`lastApplied`). Essa validação evita inconsistências no estado.
- **Execução de Comandos:** Cada entrada do log é avaliada com base no tipo de operação:
 - Para comandos **SET**, o sistema salva a chave e o valor no armazenamento.
 - Para comandos **DELETE**, a chave correspondente é removida.
- **Atualização do Estado:** Após a execução bem-sucedida de uma entrada, o índice `lastApplied` é atualizado para refletir a última entrada processada.

Essa abordagem assegura que todas as operações comprometidas sejam aplicadas de forma ordenada e consistente, mesmo em cenários de falhas.

Por exemplo, a função avalia o tipo de comando com um **switch** simples, que diferencia as operações **SET** e **DELETE**, garantindo a aplicação correta e apropriada ao

armazenamento de dados. Erros encontrados durante o processamento, como falhas no acesso ao armazenamento, são capturados e reportados, reforçando a confiabilidade do sistema.

Essa lógica de aplicação de comandos é essencial para manter a integridade do estado global do sistema e sustenta a confiabilidade do algoritmo *Raft* em garantir consistência forte entre os nós.

5.3.2.11 Benefícios da Implementação

A utilização do algoritmo *Raft* trouxe os seguintes benefícios ao projeto:

- **Consistência Forte:** Garantia de que todos os nós possuem o mesmo estado após a aplicação das entradas de log.
- **Tolerância a Falhas:** Capacidade de continuar operando mesmo com a falha de alguns nós, devido ao mecanismo de eleição e replicação.
- **Escalabilidade Horizontal:** Facilidade em adicionar novos nós ao cluster sem impactar a consistência ou disponibilidade.
- **Alta Disponibilidade:** Possibilidade de atender operações de leitura em qualquer nó e rápida recuperação de falhas.

5.3.3 Definição das Interfaces com Protocol Buffers

Para facilitar a comunicação entre os componentes do sistema distribuído e garantir interoperabilidade, utilizamos o *Protocol Buffers* (protobuf), uma linguagem neutra e independente de plataforma para serializar dados estruturados. O protobuf permite definir interfaces de serviços e estruturas de dados de forma concisa, gerando código automaticamente para várias linguagens de programação.

5.3.3.0.1 Arquivo `raft.proto`

O arquivo `raft.proto` define as interfaces e mensagens necessárias para implementar o serviço **Raft**, que é fundamental para a operação do protocolo de consenso Raft no sistema distribuído. Ele especifica os métodos que permitem a comunicação entre os nós do cluster para manter a consistência e coordenar ações críticas como eleições e replicação de logs.

Os métodos definidos incluem:

- **RequestVote:** Gerencia solicitações de voto durante o processo de eleição de um novo líder. Um nó candidato utiliza este método para solicitar votos de outros nós.

- **Heartbeat**: Realiza a sincronização periódica entre o líder e os seguidores, mantendo-os atualizados sobre o estado atual do sistema e assegurando que os nós seguidores não iniciem eleições desnecessárias.
- **SearchLog**: Permite a busca por entradas específicas no log replicado, facilitando a recuperação e verificação de informações armazenadas.

Cada um desses métodos é acompanhado por mensagens que encapsulam os parâmetros de entrada e as respostas necessárias para o funcionamento correto do protocolo. Por exemplo:

- **RequestVoteArgs** e **RequestVoteReply**: Estruturas que contêm informações sobre o termo atual, o candidato solicitante e se o voto foi concedido, respectivamente. Isso permite aos nós decidirem se devem votar em um candidato com base nos termos e no histórico de logs.
- **HeartbeatArgs** e **HeartbeatReply**: Mensagens que incluem informações sobre o termo, o identificador do líder, entradas de log a serem replicadas, e confirmam o sucesso da operação. Isso garante que os seguidores atualizem seus logs e termos conforme necessário.
- **LogEntry** e **LogCMD**: Estruturas que representam entradas individuais no log, incluindo o termo, índice e o comando associado (como operações de **SET** ou **DELETE**). Essas estruturas são fundamentais para a replicação consistente de logs entre os nós.
- **SearchLogArgs** e **SearchLogReply**: Mensagens que permitem a busca por entradas no log com base em critérios específicos, retornando o comando encontrado, se disponível, e informações sobre seu estado de commit.

A definição clara dessas mensagens e serviços no arquivo `raft.proto` é essencial para o funcionamento correto do protocolo Raft no sistema. Ela estabelece um contrato de comunicação entre os nós, garantindo que todos utilizem os mesmos formatos de mensagem e compreendam os mesmos parâmetros e respostas.

Além disso, ao utilizar o Protocol Buffers para definir essas interfaces, o sistema se beneficia de uma comunicação eficiente e serialização compacta, o que é crítico para desempenho em sistemas distribuídos. A geração automática de código a partir dos arquivos `.proto` também assegura consistência entre as implementações dos nós e facilita a manutenção e evolução do sistema.

5.3.3.1 Geração de Código com *protoc*

Utilizando o compilador `protoc`, os arquivos `.proto` são processados para gerar código em Go. As seguintes etapas foram realizadas:

1. Definição dos arquivos `database.proto` e `raft.proto`, especificando serviços, métodos e mensagens.
2. Execução do comando `protoc` para gerar os códigos:

```
protoc --go_out=. --go-grpc_out=. proto/database.proto
protoc --go_out=. --go-grpc_out=. proto/raft.proto
```

3. Inclusão dos pacotes gerados no projeto, permitindo que a aplicação utilize as interfaces definidas.

5.3.3.2 Integração interna

Os códigos gerados automaticamente fornecem as estruturas e interfaces necessárias para implementar os servidores e clientes gRPC. No lado do servidor, as interfaces são implementadas para atender às chamadas dos clientes, enquanto no lado do cliente, as funções geradas permitem realizar chamadas remotas aos métodos definidos.

5.3.3.2.1 Implementação do Servidor

No servidor, as interfaces geradas são implementadas conforme a lógica do sistema. Por exemplo, para o serviço `Database`, os métodos `Set`, `Delete` e `Get` são implementados para interagir com o armazenamento local e coordenar a replicação via Raft.

5.3.3.2.2 Implementação do Cliente

No cliente, as funções geradas são utilizadas para criar conexões e realizar chamadas aos métodos remotos. Isso permite que componentes do sistema ou aplicativos externos interajam com o banco de dados distribuído de forma transparente.

5.3.3.3 Benefícios da Utilização do Protobuf

A adoção do *Protocol Buffers* trouxe diversos benefícios para o projeto:

- **Eficiência:** As mensagens protobuf são compactas, reduzindo o consumo de banda e melhorando a performance.

- **Interoperabilidade:** As definições em `.proto` são independentes de linguagem, permitindo futuras extensões em outras linguagens se necessário.
- **Facilidade de Manutenção:** As mudanças nas interfaces podem ser gerenciadas centralmente nos arquivos `.proto`, e o código gerado reflete automaticamente essas mudanças.
- **Consistência:** Garante que todos os componentes do sistema utilizem as mesmas definições de mensagens e serviços, evitando incompatibilidades.

5.3.4 Módulo *grpcutil*

O pacote `grpcutil` é responsável por abstrair a comunicação entre os nós do sistema utilizando o protocolo gRPC. Esse módulo encapsula tanto a funcionalidade de clientes quanto de mocks, permitindo maior flexibilidade nos testes e na execução da aplicação. Abaixo, explicamos os principais componentes do pacote:

5.3.4.1 Cliente gRPC

A implementação do cliente gRPC, representada pela estrutura `grpcClient`, fornece métodos para interagir com outros nós no cluster, incluindo as chamadas `Heartbeat` e `RequestVote`. Esses métodos são essenciais para o funcionamento do protocolo Raft, pois possibilitam: - **Heartbeat:** A comunicação periódica do nó líder com os seguidores para manter a consistência e verificar se eles estão ativos. - **RequestVote:** O processo de eleição de um novo líder quando o atual está inativo ou ocorre um reinício.

O cliente é configurado com as seguintes características: - Utilização de credenciais inseguras (`insecure.NewCredentials()`), simplificando o desenvolvimento e os testes. - Manuseio automático da conexão com o servidor gRPC via `grpc.DialOption`.

Além disso, a função `makeGrpcClient` facilita a criação de instâncias do cliente com os parâmetros necessários para a comunicação.

5.3.4.2 Configuração do Cliente

O módulo também inclui uma estrutura `config` que permite ativar um cliente mock por meio de uma variável de ambiente (`MOCK_GRPC_CLIENT`). Esse mecanismo possibilita alternar entre o cliente real e um cliente simulado para cenários de testes, sem necessidade de alterar o código principal.

A função `MakeClient` atua como uma fábrica que decide qual implementação (real ou mock) será utilizada com base nessa configuração, garantindo flexibilidade no ambiente de execução.

5.3.4.3 Cliente Mock

O `mockClient` é uma implementação simulada do cliente gRPC, utilizada principalmente para testes. Ele é controlado por um `mockController`, que permite configurar as respostas esperadas para as chamadas `Heartbeat` e `RequestVote`. Isso é especialmente útil para validar a lógica de negócios do algoritmo Raft em situações controladas, como: - Simular falhas na comunicação. - Testar cenários em que o líder ou os seguidores retornam respostas específicas.

O uso de mocks reduz a dependência de um ambiente distribuído completo durante o desenvolvimento e os testes iniciais, acelerando o ciclo de desenvolvimento.

5.4 Testes e Avaliação

Nesta seção, são apresentados os procedimentos de teste desenvolvidos para avaliar o funcionamento do sistema distribuído de banco de dados chave-valor. O objetivo é comprovar, de maneira prática e sistemática, o atendimento aos requisitos funcionais (armazenar, recuperar, atualizar e deletar dados) e não funcionais (escalabilidade horizontal, tolerância a falhas, consistência e observabilidade).

5.4.1 Estratégia de Teste

A estratégia de testes adotada neste projeto foi planejada para cobrir tanto o atendimento dos requisitos funcionais quanto o cumprimento dos requisitos não funcionais. Assim, cada categoria de teste foi escolhida de forma a validar aspectos específicos do sistema, conforme descrito a seguir:

- **Testes Funcionais (CRUD):** Os testes de *POST* (armazenamento), *GET* (recuperação), *PUT* (atualização) e *DELETE* (remoção) foram concebidos para comprovar que o sistema satisfaz o requisito fundamental de operar como um banco de dados chave-valor. Esses testes garantem que as funcionalidades básicas estejam corretas, incluindo a capacidade de lidar com valores de grande porte (como documentos PDF e *strings* extensas).
- **Testes de Consistência e Eleição (Raft):** A estratégia de eleger um líder e, posteriormente, promover uma nova eleição (desligando o nó líder) confirma a robustez do algoritmo Raft no contexto de tolerância a falhas. Esses testes mostram que o cluster permanece funcional mesmo diante de paradas inesperadas de nós, atendendo aos requisitos de consistência e alta disponibilidade. - A verificação dos *logs* de console e da replicação de entradas (*WAL files*) mostra como as escritas são registradas e como ocorre a convergência para um único líder.

- **Testes de Desempenho com Inserções (Foco no Nó Líder):** - As inserções (*POST*) foram agrupadas e enviadas em lote para avaliar a capacidade do nó líder de processar múltiplas requisições em um curto intervalo de tempo. - Como o líder concentra as operações de gravação, esse teste também visa verificar se a replicação *gRPC* para os outros nós (seguidores) impacta significativamente no *throughput*. - Problemas observados em alta taxa de inserção (erros *500 Internal Server Error*) reforçam a necessidade de controle de taxa (*rate limiting*) ou expansão de recursos, evidenciando que esse requisito de **desempenho** precisa ser gerenciado de acordo com a carga esperada.
- **Teste de 1000 Requisições *GET*:** - Esse teste foi idealizado para validar a **escalabilidade em leituras** e também demonstrar a **performance** do sistema sob alto volume de consultas. - As leituras são distribuídas entre os nós, o que potencialmente reduz a carga em cada instância quando escalado horizontalmente.
- **Teste de Armazenamento e Recuperação de PDF:** - Além de comprovar a capacidade de gravar e recuperar valores grandes (convertidos em **Base64**), esse teste demonstra a aplicabilidade prática do sistema em cenários que envolvem arquivos binários. - Dessa forma, evidencia-se a aderência aos requisitos funcionais de armazenamento e leitura de dados e a robustez na manipulação de documentos de maior porte.

Em conjunto, esses testes atestam o atendimento das principais diretrizes funcionais e não funcionais estabelecidas no capítulo de requisitos, especialmente no que diz respeito à:

- **Escalabilidade e Desempenho:** verificados por meio do teste de 1000 *GET* em alta cadência e do teste de múltiplas inserções para avaliar o desempenho do nó líder.
- **Consistência e Tolerância a Falhas:** garantidas pelos testes de eleição e reeleição de nós, usando Raft, assegurando que apenas um líder vigore por termo e que as réplicas de log mantenham o estado sincronizado.
- **Robustez na Manipulação de Dados Grandes:** demonstrada pelos testes que inserem *strings* extensas (2.106.540 caracteres) e arquivos PDF em **Base64**.

Assim, a estratégia de teste abrange desde a verificação de funcionalidades básicas (CRUD) até a validação de cenários de alto volume de leitura e taxas elevadas de escrita, demonstrando como cada conjunto de testes contribui diretamente para confirmar os requisitos de escalabilidade, disponibilidade, consistência e desempenho de um banco de dados chave-valor distribuído.

5.4.2 Plano de Testes

5.4.2.1 Testes Funcionais do Banco de Dados

- **POST (Armazenamento de Dados)**

Objetivo: Validar se o sistema aceita dados de grande tamanho (em formato `string`) associados a uma chave textual.

Procedimento:

1. Realizar uma requisição `POST` para a API REST, informando (`chave`, `valor`) no corpo da mensagem.
2. Verificar o retorno de sucesso (código `201 Created`).
3. Executar uma requisição `GET` para a mesma chave, a fim de confirmar se o valor foi armazenado corretamente.

Evidência de Teste:

- A Figura 8 apresenta a requisição `POST` com o corpo contendo um `value` de tamanho significativo, neste caso com 2.106.540 caracteres, associado à chave “arquivo gigante”. - A Figura 9 exibe uma captura de tela do *Notepad* do Windows 11, evidenciando a contagem de caracteres do arquivo antes de ser enviado ao sistema. - Ao verificar a resposta do servidor, constatou-se que o serviço retornou `201 Created`, indicando sucesso na criação do recurso. Além disso, o `index` retornado (16) confirma que esta é a décima sexta chave armazenada no sistema, e o atributo `noted = true` sinaliza que a gravação foi concluída corretamente.

- **GET (Recuperação de Dados)**

Objetivo: Conferir se o sistema retorna corretamente o valor (decodificado) ao receber uma chave, validando a persistência.

Procedimento:

1. Após realizar o `POST` de um `value` extenso (Figura 8), chamar o método `GET` na mesma chave (por exemplo, “arquivo-gigante”).
2. Verificar se a resposta contém exatamente o mesmo conteúdo armazenado previamente.
3. Observar se o código de retorno é `200 OK`.

Evidência de Teste:

- A Figura 10 mostra a resposta obtida ao chamar `GET` para a chave “arquivo-gigante”, incluindo o conteúdo retornado e o código `200 OK`.

Comentários e Observações:

- O `GET` confirma que o conteúdo armazenado é idêntico ao enviado, mostrando a

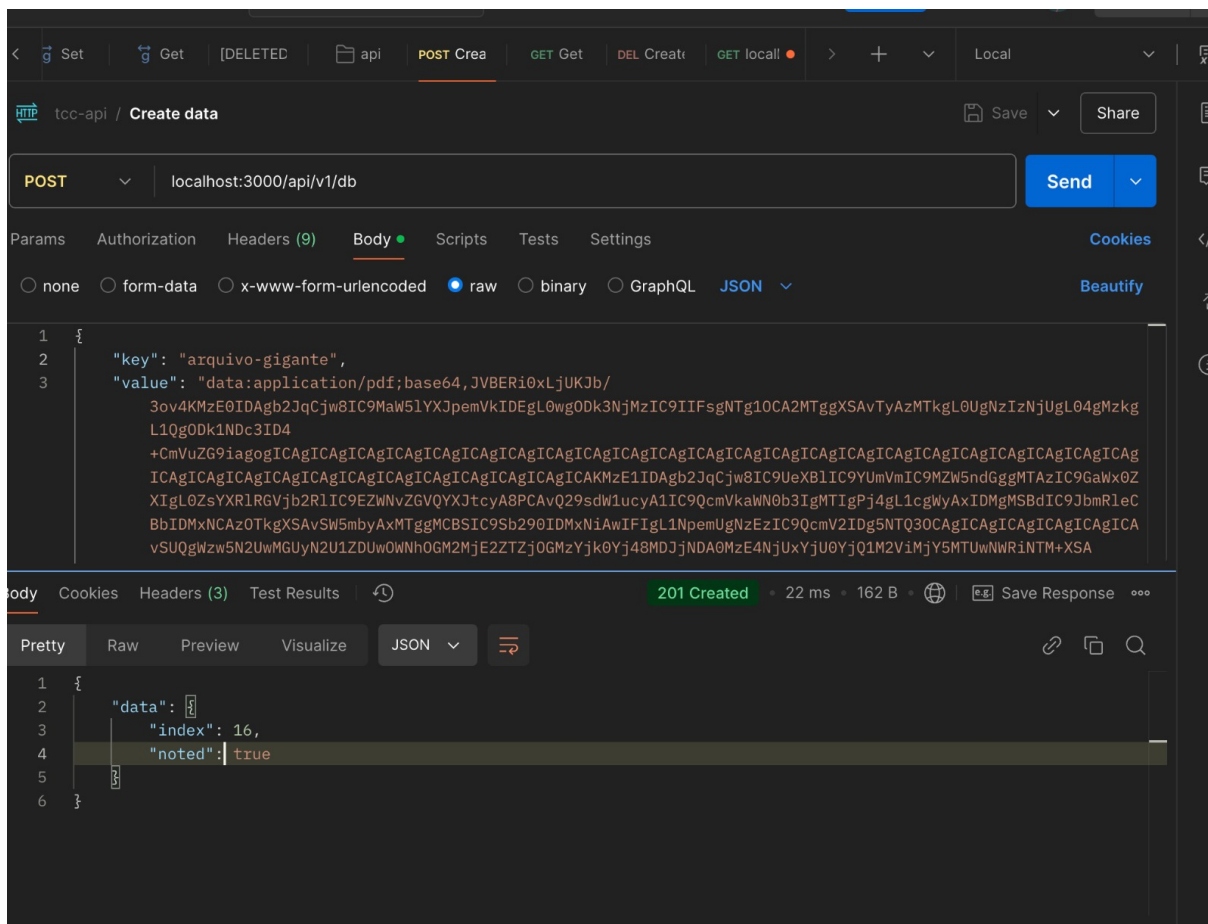


Figura 8 – Exemplo de requisição POST para salvar um valor extenso no sistema.

robustez do sistema para lidar com grandes strings. - A resposta 200 OK sinaliza sucesso na recuperação.

- UPDATE (Atualização de Dados)

Objetivo: Garantir que dados já existentes possam ser sobrescritos com novos valores.

Procedimento:

1. Realizar **POST** inicial para a chave “**arquivo-gigante**” (como no teste de **SET**).
2. Executar uma requisição **PUT** na mesma chave (“**arquivo-gigante**”) com um **value** diferente (por exemplo, “somente uma palavra simples”).
3. Chamar **GET** novamente e checar se o valor retornado corresponde ao **value** atualizado.

Evidência de Teste:

- A Figura 11 exibe a requisição PUT realizada, onde a mesma chave “arquivo-gigante” recebeu um novo conteúdo. - Na sequência, a Figura 12 mostra o GET retornando o conteúdo atualizado, confirmando a sobrescrita dos dados.

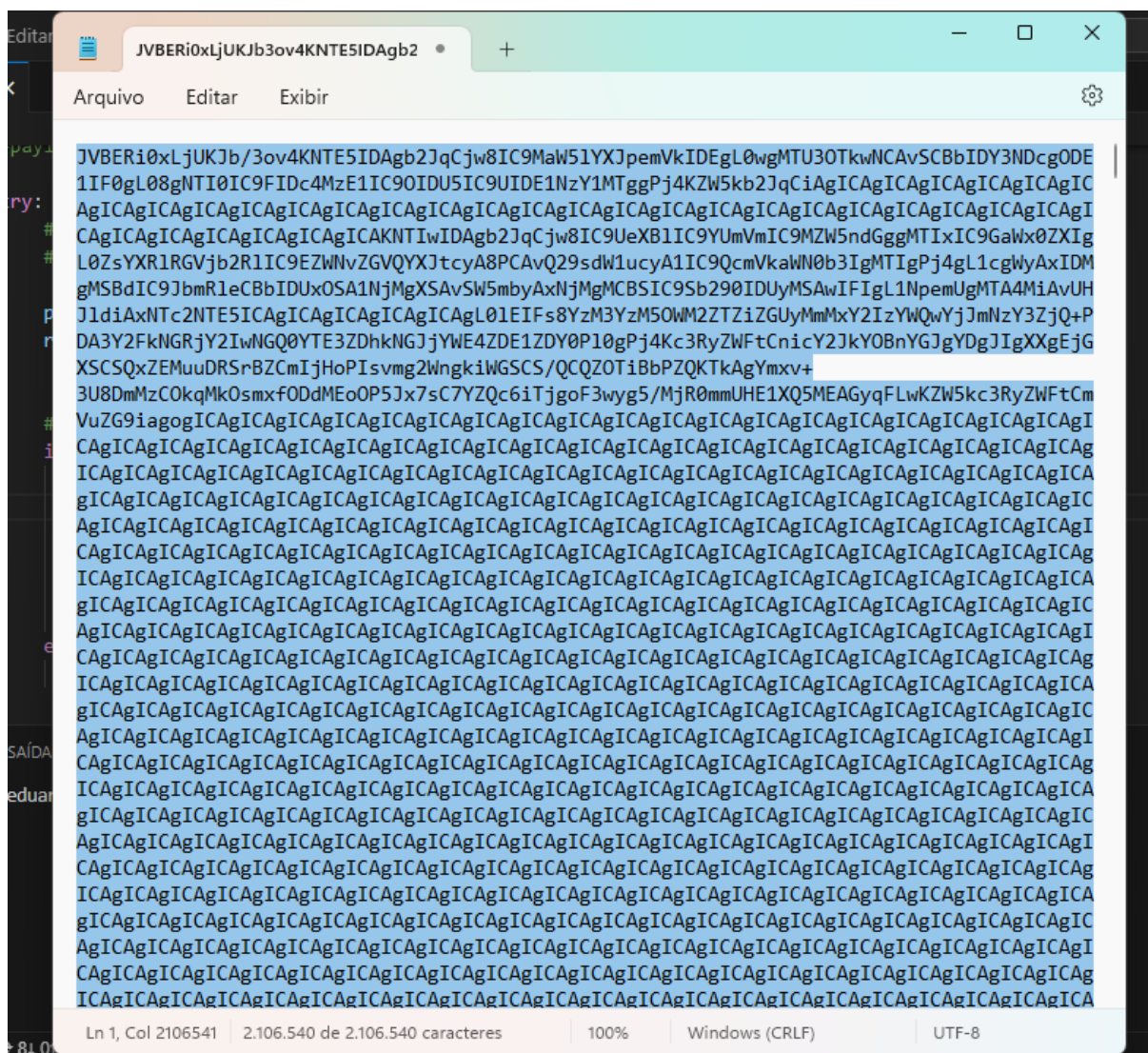


Figura 9 – Contagem de 2.106.540 caracteres do value, conforme verificado no Notepad.

Comentários e Observações:

- O teste comprova que o sistema permite substituir integralmente valores em uma chave existente. - A confirmação via GET atesta a consistência do banco de dados após a atualização.

- **DELETE (Remoção de Dados) Objetivo:** Verificar se o sistema remove corretamente o conteúdo associado a uma chave.

Procedimento:

1. Enviar SET para criar um registro associado a uma chave (por exemplo, “arquivo-gigante”).
2. Enviar DELETE para a mesma chave.
3. Tentar GET para confirmar que a chave não existe mais.

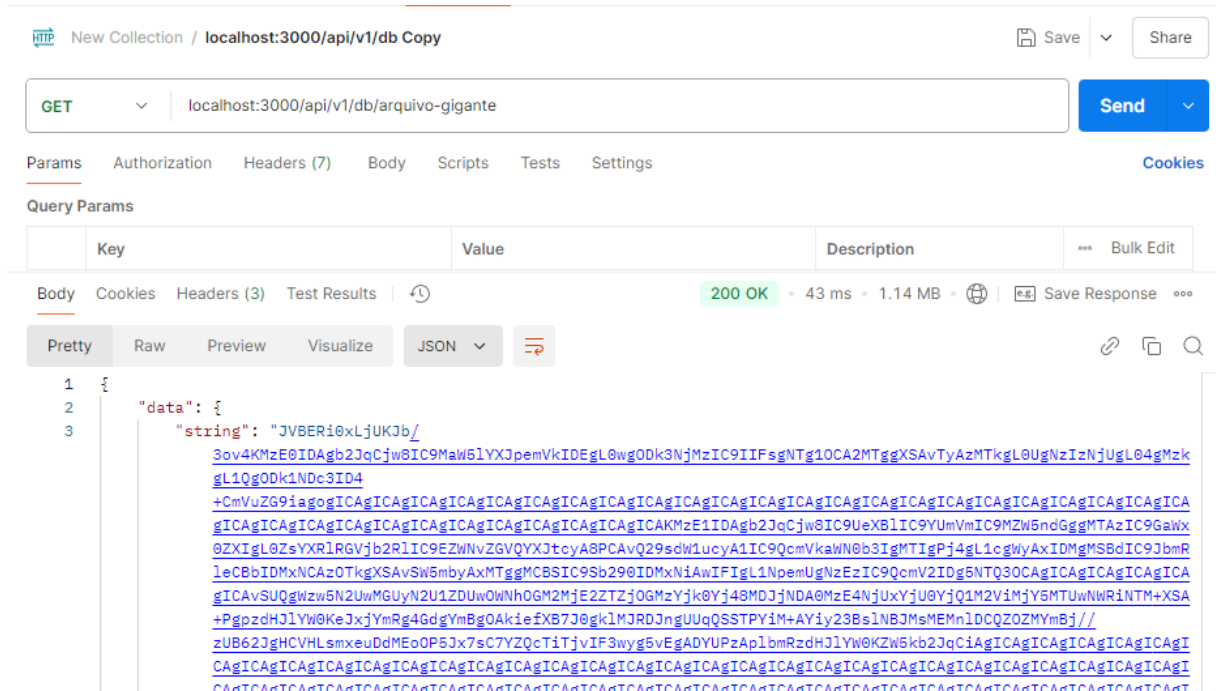


Figura 10 – Valor recuperado do sistema via GET, confirmando a persistência.

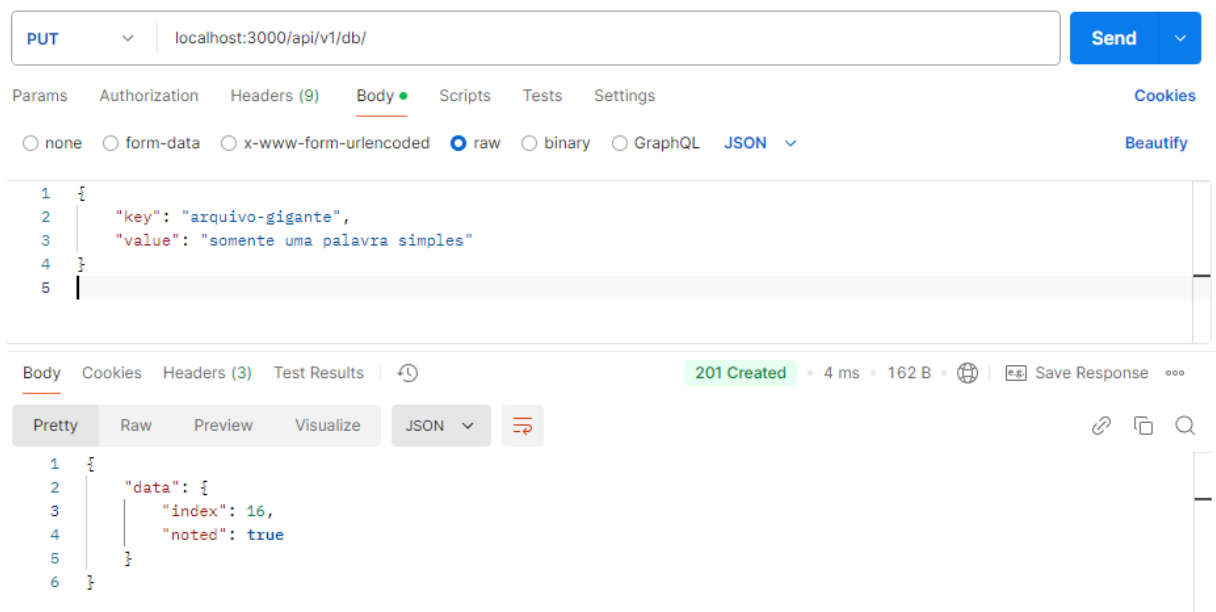


Figura 11 – Atualização via HTTP PUT para a chave “arquivo-gigante”.

Evidência de Teste:

A Figura 13 ilustra a requisição HTTP DELETE feita para a chave “arquivo-gigante”, retornando um **status code** 204, o que significa que a remoção foi concluída com sucesso. Na sequência, a Figura 14 mostra a tentativa de HTTP GET para a mesma chave, produzindo a mensagem de erro "pebble: not found", indicando que o valor anteriormente armazenado não está disponível, conforme esperado após a deleção.

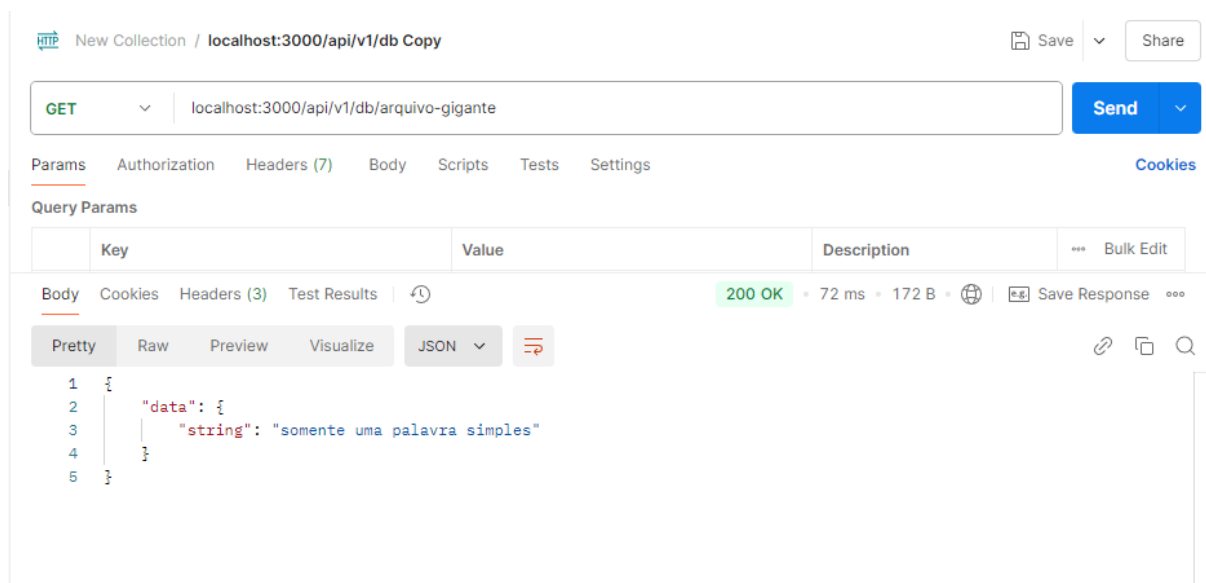


Figura 12 – Nova requisição HTTP GET, demonstrando o valor atualizado.

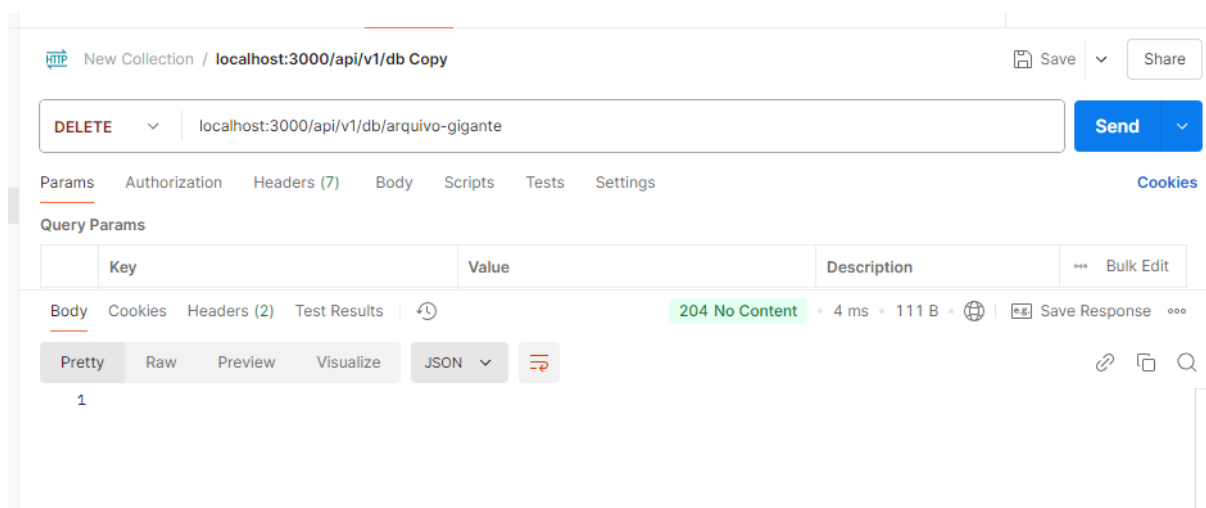


Figura 13 – Remoção do registro via HTTP DELETE para a chave “arquivo-gigante”.

5.4.2.2 Testes de Consistência e Eleição (Raft)

- **Casos de Sucesso em Eleição Objetivo:** Validar a escolha de um líder quando o cluster inicia ou fica sem líder.

Procedimento:

1. Iniciar o cluster com três nós (A, B e C).
2. Observar os logs do console para confirmar a eleição de um líder.

Evidência de Teste:

A Figura 15 mostra os registros de *console* logo após o cluster ser iniciado. É possível verificar que o nó C dispara uma mensagem de **Timed out, starting**

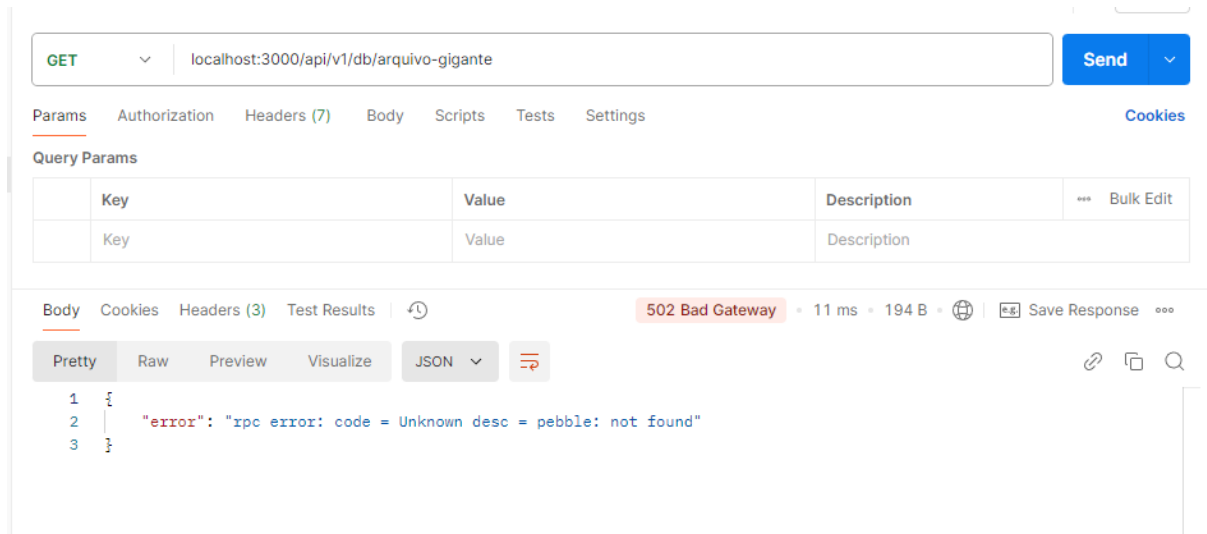


Figura 14 – Tentativa de HTTP GET após a remoção, evidenciando que o valor não existe mais.

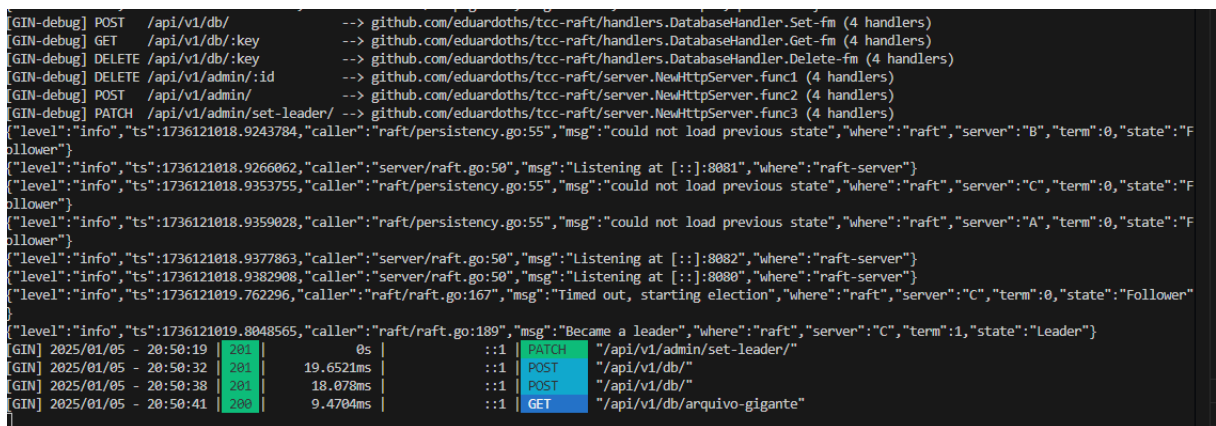


Figura 15 – Logs da aplicação ao subir o cluster e iniciar o processo de eleição.

election, indicando que ele se tornou *Candidate* após não receber *heartbeats* de um líder. Na sequência, o mesmo nó registra a mensagem *Became a leader* ("Became a leader", "server": "C", "term": 1, "state": "Leader"), confirmando que obteve votos suficientes.

Outro ponto observado é a requisição PATCH `/api/v1/admin/set-leader/`, que é enviada para informar ao restante do sistema que o nó C foi eleito líder. Dessa forma, todos os nós passam a reconhecer C como responsável pelo gerenciamento das entradas de log e coordenação de futuras replicações.

- **Eleição de um Novo Líder Objetivo:** Verificar se o cluster permanece funcional em cenário de troca de liderança.

Procedimento:

1. Iniciar o cluster com múltiplos nós (A, B e C) e observar a eleição inicial do

líder.

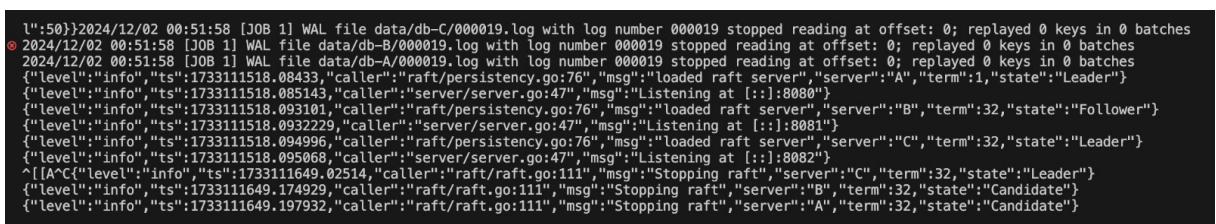
2. Verificar a ocorrência de uma nova eleição pela parada explícita de um nó líder).
3. Confirmar que um novo líder assume o papel.

Evidência de Teste:

A Figura 16 exibe o registro de console (*terminal*) durante o surgimento de uma nova eleição e a recuperação dos nós. Abaixo, destacam-se os principais pontos dos logs:

- 2024/12/02 00:51:58 [JOB 1] WAL file data/db-C/000019.log ... replayed 0 keys: Indica o carregamento inicial dos arquivos de log (WAL) para cada nó (A, B e C). É possível ver que eles não possuem entradas de log pendentes para replay, pois o offset permanece em zero.
- "msg":"loaded raft server","server":"A","term":1,"state":"Leader": O nó A inicia como líder no termo 1. Já o nó B inicia como Follower no termo 32 e o nó C também é reportado como Leader no termo 32, o que sugere um cenário de inconsistência temporária em seus estados antes de qualquer sincronização.
- "msg":"Stopping raft","server":"C","term":32,"state":"Leader": Posteriormente, o nó C, que também se via como líder, é parado, provocando a necessidade de uma nova eleição para que apenas um líder fique ativo.
- "msg":"Stopping raft","server":"B","term":32,"state":"Candidate" e "msg":"Stopping raft","server":"A","term":32,"state":"Candidate": Similarmente, as paradas dos nós B e A são reportadas, confirmando que ambos se encontram em transição de estados (por exemplo, Candidate antes de se desligarem efetivamente).

Durante esse processo, o protocolo Raft garante que apenas um nó atue como líder válido em cada termo. Assim que o nó líder final é definido, as solicitações de SET e GET permanecem disponíveis para o cliente, pois os nós seguidores conseguem redirecionar pedidos ao novo líder ou atualizar-se quando retornam ao cluster.



```
l":50}}2024/12/02 00:51:58 [JOB 1] WAL file data/db-C/000019.log with log number 000019 stopped reading at offset: 0; replayed 0 keys in 0 batches
2024/12/02 00:51:58 [JOB 1] WAL file data/db-B/000019.log with log number 000019 stopped reading at offset: 0; replayed 0 keys in 0 batches
2024/12/02 00:51:58 [JOB 1] WAL file data/db-A/000019.log with log number 000019 stopped reading at offset: 0; replayed 0 keys in 0 batches
{"level":"info","ts":1733111518.00433,"caller":"raft/persistence.go:76","msg":"loaded raft server","server":"A","term":1,"state":"Leader"}
{"level":"info","ts":1733111518.005143,"caller":"server/server.go:47","msg":"Listening at [:]:8080"}
{"level":"info","ts":1733111518.003101,"caller":"raft/persistence.go:76","msg":"loaded raft server","server":"B","term":32,"state":"Follower"}
{"level":"info","ts":1733111518.0032229,"caller":"server/server.go:47","msg":"Listening at [:]:8081"}
{"level":"info","ts":1733111518.004996,"caller":"raft/persistence.go:76","msg":"loaded raft server","server":"C","term":32,"state":"Leader"}
{"level":"info","ts":1733111518.005068,"caller":"server/server.go:47","msg":"Listening at [:]:8082"}
^[[A^C{"level":"info","ts":1733111649.02514,"caller":"raft/raft.go:111","msg":"Stopping raft","server":"C","term":32,"state":"Leader"}
{"level":"info","ts":1733111649.174929,"caller":"raft/raft.go:111","msg":"Stopping raft","server":"B","term":32,"state":"Candidate"}
{"level":"info","ts":1733111649.197932,"caller":"raft/raft.go:111","msg":"Stopping raft","server":"A","term":32,"state":"Candidate"}
```

Figura 16 – Terminal do serviço durante uma nova eleição, exibindo paradas de nós e reeleições.

5.4.2.3 Testes de Consistência de Logs

```

Resultados das inserções:
1: Key=ApC1, Value=ax8j4gYX3ZtvCyymCz, Index=1
2: Key=2Q5C2, Value=DX7tg0Y2xtk0pc4Nav, Index=2
3: Key=UhfYoC3, Value=1lMxbl7r-sy8asz5c2KD, Index=3
4: Key=186C4, Value=vQh2i5oH964WwomrvQ0, Index=4
5: Key=ro6C5, Value=h6qd27V1yHLLer7s, Index=5
6: Key=clC6, Value=FLIqWPL4Dkjw0RRzBo, Index=6
7: Key=Br0egC7, Value=DsYmPhS0ndNcAXzeEX, Index=7
8: Key=ac0WC8, Value=7wuldjHwdimLwa1P2K3j, Index=8
9: Key=RdLHic9, Value=HmpfKn6apdu6V3Z, Index=9
10: Key=susvC10, Value=5QGvSLEnug05ckP, Index=10
11: Key=5BAFEC11, Value=ejH8wJNRppshNvXk, Index=11
12: Key=RC12, Value=BuFRnvgXAHnOh640vLt, Index=12
13: Key=RLvC13, Value=6oXOXmmAMRqieIHafeo0, Index=13
14: Key=wUvC14, Value=GfGKLWtWGRzHB050, Index=14
15: Key=47DCgC15, Value=HbMzIGW0GMIEFP70, Index=15
16: Key=xwq2YC16, Value=OC39fiS9s25xMCMG5Jfc, Index=16
17: Key=xmC17, Value=re8MjUYaVvLNMFO9, Index=17
18: Key=aPfc18, Value=ph9WBJhJG18jWu38, Index=18
19: Key=3mCaC19, Value=LSRtI8cxb4siisMKyk, Index=19
20: Key=rWj0C20, Value=3vRF2VavMjbGw6HxbV, Index=20
21: Key=sehC21, Value=8LHbv3jLBdIy22CFo2, Index=21
22: Key=qRhVC22, Value=UXPk0UgBKD8Um8LvF, Index=22
23: Key=6OC23, Value=tVwp9UCRN0KGFpRmDC, Index=23
24: Key=CSYxC24, Value=j2IzY1xjUmveIuPUQt, Index=24
25: Key=b9GUC25, Value=JENPGDnIToLT918Jg, Index=25
26: Key=sLC26, Value=Lwj8MfK0dnVYfBg71, Index=26
27: Key=Se7C27, Value=DZxvXqfeF9cGMky, Index=27
28: Key=8A4jC28, Value=SyUwyELHun3G2CHMHD, Index=28
29: Key=pkC29, Value=7b7MPGHx2hdK2JnUlus, Index=29
30: Key=rVBuJC30, Value=fCwMts5eYuuQuD, Index=30
31: Key=rnyk7C31, Value=UZE4LfxhYyPp, Index=31
32: Key=qLTW6C32, Value=sFLYXG3ELGUW5Lw, Index=32
33: Key=mN2xC33, Value=wEzMthB0L0jJcolv, Index=33
34: Key=j0WwC34, Value=q1Kuscq6Q5pRn0Wvgc1, Index=34
35: Key=5t9C35, Value=90ajjHXFZep31NH0co, Index=35
36: Key=n5oWC36, Value=vYt9NunEeyT7ghzKly, Index=36
37: Key=2duC37, Value=Vt0TNNunEeyT7ghzKly, Index=37
38: Key=5TC38, Value=CBtgViMacdUws2WQk, Index=38
39: Key=dLC39, Value=LxTC1d1259TN21g30w7U, Index=39
40: Key=9LRC40, Value=K0mSuLUvdXuJt0kVrG, Index=40
41: Key=QVGC41, Value=nZID0jgbQcJx93PL, Index=41
42: Key=FDjC42, Value=J9gxEGtXL7BwE2ShN, Index=42
43: Key=no2C43, Value=2F2Z2ZSdpXixgntEeYj, Index=43
44: Key=F64iC44, Value=dNYzSEbxSwjYhPP, Index=44
45: Key=Ub53jC45, Value=jn7SBxBcBUYkyw4dN, Index=45
46: Key=lfrgC46, Value=WsDfS07XdrpTqn9zG, Index=46
47: Key=PUizC47, Value=Fa8o37eQznZR42j, Index=47
48: Key=tzUC48, Value=5N55S9pQjd32pQELXk, Index=48
49: Key=mFp6pC49, Value=Zd0CCjZwzK5K7M2UDDH2, Index=49
50: Key=CnQuiC50, Value=DYeaQ0rJIoL8KPAc, Index=50
51: Key=HBC51, Value=gQGgIal7R3vTb268b, Index=51
52: Key=ApiC52, Value=IweYu8TGcAMk62v6GGmk, Index=52
53: Key=qKMC53, Value=rAgsGvnQKFAVSeKw, Index=53
54: Key=SRc54, Value=AD18xpLmZziurLBEHk, Index=54
55: Key=ecCa55, Value=xVUumB1xmP21gUc, Index=55
56: Key=uvTBC56, Value=kLstYPSOLRvgiwsvmhs, Index=56
57: Key=pFQxC57, Value=7cAuIcz32KlxbqleI, Index=57
58: Key=q90C58, Value=vVQ0Wdb5ANyJcXn, Index=58
59: Key=YXfAx59, Value=VLGts5UkMhuUkUj, Index=59
60: Key=HUFBbC60, Value=76TN7ensRcTPhNJ, Index=60
Total de inserções realizadas com sucesso: 60

```

Figura 17 – Terminal exibindo a execução de um script em Python que envia requisições POST ao sistema.

- **Replicação Correta dos Logs Objetivo:** Verificar se as operações de escrita POST são processadas corretamente e se o sistema lida adequadamente com múltiplas inserções em sequência.

Procedimento:

1. Elaborar um script em Python para enviar **60 requisições POST** em um intervalo de **1 minuto**.
2. Acompanhar o *output* gerado pelo script (Figura 17) e os *logs* internos do serviço (Figura 18), verificando se cada requisição resultou em 201 Created.
3. Validar manualmente, por meio de GET, alguns dos dados enviados, confirmando que o sistema efetivamente armazenou as chaves e valores.

Evidência de Teste:

- A Figura 17 mostra o *script* Python em execução, listando 60 envios de

[GIN]	2025/01/05 - 23:46:12	201	6.7544ms	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:13	201	1.5814ms	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:14	201	1.2113ms	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:15	201	2.09ms	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:16	201	1.7112ms	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:17	201	1.6911ms	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:18	201	60µs	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:20	201	617.3µs	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:21	201	594.6µs	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:22	201	622.2µs	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:23	201	2.2991ms	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:24	201	1.812ms	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:25	201	2.4477ms	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:26	201	1.1395ms	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:27	201	1.7273ms	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:28	201	1.3015ms	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:29	201	2.0278ms	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:30	201	251µs	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:31	201	1.3727ms	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:32	201	504.5µs	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:33	201	1.1661ms	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:34	201	1.0909ms	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:35	201	1.1884ms	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:36	201	873µs	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:37	201	506.7µs	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:38	201	1.8879ms	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:39	201	1.3793ms	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:40	201	2.1052ms	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:41	201	505.7µs	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:42	201	1.0869ms	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:43	201	515.8µs	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:44	201	1.157ms	::1	POST	"/api/v1/db/"
[GIN]	2025/01/05 - 23:46:45	201	1.0823ms	::1	POST	"/api/v1/db/"

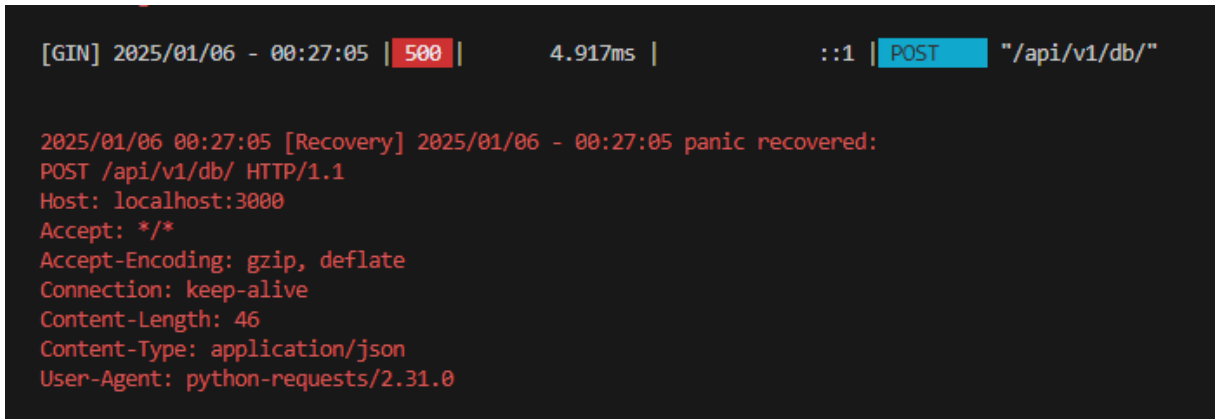
Figura 18 – Logs internos do serviço, evidenciando o recebimento das requisições e o retorno de 201 Created.

chave/valor sem falhas. - A Figura 18 evidencia que o serviço recebeu cada requisição, retornando 201 Created para confirmar o armazenamento. - Na checagem manual via GET, todos os registros retornaram o valor correto, indicando que a escrita foi concluída com sucesso. - A média simples de tempo de resposta aproximou-se de **1215,18 ms**, evidenciando que o sistema conseguiu lidar com as requisições em lote dentro de um intervalo razoável.

5.4.2.4 Limitação de Taxa e Erros em Altas Taxas de Inserção

Durante os testes com o *script* em Python, observou-se que o envio de requisições em uma ****taxa muito alta**** (por exemplo, cerca de 1000 POST em 10 segundos) resultou em falhas na inserção dos dados. O nó líder, responsável por coordenar as escritas e responder aos clientes, não conseguia processar as solicitações em um intervalo inferior a 1 segundo na maioria das vezes.

Como consequência, houve acúmulo de requisições pendentes, fazendo com que



```
[GIN] 2025/01/06 - 00:27:05 | 500 | 4.917ms | ::1 | POST "/api/v1/db/"

2025/01/06 00:27:05 [Recovery] 2025/01/06 - 00:27:05 panic recovered:
POST /api/v1/db/ HTTP/1.1
Host: localhost:3000
Accept: */*
Accept-Encoding: gzip, deflate
Connection: keep-alive
Content-Length: 46
Content-Type: application/json
User-Agent: python-requests/2.31.0
```

Figura 19 – 500 Internal Server Error retornado pelo servidor durante a sobrecarga de requisições.

algumas inserções não fossem concluídas e gerando erros no servidor. A Figura 19 ilustra o 500 Internal Server Error exibido no terminal, evidenciando que o sistema não conseguiu lidar com o grande volume de POST em um intervalo muito curto.

Para contornar esse problema, foi introduzida uma trava de apenas *uma requisição por segundo*, o que permitiu ao nó líder processar o fluxo de POST de maneira mais previsível, reduzindo ou eliminando falhas na inserção. Essa observação reforça a importância de avaliar o *limite de throughput* do sistema e de aplicar mecanismos de controle de taxa (rate limiting) em aplicações que exigem alta cadência de gravações.

5.4.2.5 Teste de Carga com 1000 Requisições GET

Para avaliar o desempenho e a confiabilidade do serviço ao lidar com múltiplas operações de leitura em um curto intervalo de tempo, foi desenvolvido um *script* em Python capaz de enviar **1000 requisições GET** simultâneas (ou em rápida sucessão) para o banco de dados distribuído. O objetivo foi verificar se o sistema retornava as chaves e os valores esperados sem apresentar falhas ou degradação significativa.

Procedimento:

1. Criar um *script* Python que realize 1000 chamadas GET para o endpoint do serviço, cada requisição buscando dados distintos ou repetindo a mesma chave de teste.
2. Registrar, para cada requisição, o **status code** e o corpo da resposta (JSON) retornado.
3. Comparar o valor recebido com o valor esperado, de modo a confirmar a correção do conteúdo para cada requisição.

Evidência de Teste:

[GIN]	2025/01/06 - 00:56:43	200	597.9µs	::1	GET	"/api/v1/db/hC1nC12"
[GIN]	2025/01/06 - 00:56:43	200	0s	::1	GET	"/api/v1/db/1sC16"
[GIN]	2025/01/06 - 00:56:43	200	538.8µs	::1	GET	"/api/v1/db/ZgAcFC18"
[GIN]	2025/01/06 - 00:56:43	200	530.3µs	::1	GET	"/api/v1/db/hC1nC12"
[GIN]	2025/01/06 - 00:56:43	200	531.3µs	::1	GET	"/api/v1/db/BxC11"
[GIN]	2025/01/06 - 00:56:43	200	0s	::1	GET	"/api/v1/db/qSC3"
[GIN]	2025/01/06 - 00:56:43	200	0s	::1	GET	"/api/v1/db/hC1nC12"
[GIN]	2025/01/06 - 00:56:43	200	80µs	::1	GET	"/api/v1/db/xt1C5"
[GIN]	2025/01/06 - 00:56:43	200	0s	::1	GET	"/api/v1/db/014bvC14"
[GIN]	2025/01/06 - 00:56:43	200	538.4µs	::1	GET	"/api/v1/db/wHC7"
[GIN]	2025/01/06 - 00:56:43	200	0s	::1	GET	"/api/v1/db/wHC7"
[GIN]	2025/01/06 - 00:56:43	200	0s	::1	GET	"/api/v1/db/xt1C5"
[GIN]	2025/01/06 - 00:56:43	200	57.2µs	::1	GET	"/api/v1/db/U7f9C9"
[GIN]	2025/01/06 - 00:56:43	200	0s	::1	GET	"/api/v1/db/1sC16"
[GIN]	2025/01/06 - 00:56:43	200	54.7µs	::1	GET	"/api/v1/db/4WUC8"
[GIN]	2025/01/06 - 00:56:43	200	0s	::1	GET	"/api/v1/db/sQA1C13"
[GIN]	2025/01/06 - 00:56:43	200	0s	::1	GET	"/api/v1/db/sQA1C13"
[GIN]	2025/01/06 - 00:56:43	200	0s	::1	GET	"/api/v1/db/1sC16"
[GIN]	2025/01/06 - 00:56:43	200	0s	::1	GET	"/api/v1/db/E2FC17"
[GIN]	2025/01/06 - 00:56:43	200	0s	::1	GET	"/api/v1/db/1sC16"
[GIN]	2025/01/06 - 00:56:43	200	0s	::1	GET	"/api/v1/db/wHC7"
[GIN]	2025/01/06 - 00:56:43	200	0s	::1	GET	"/api/v1/db/xt1C5"
[GIN]	2025/01/06 - 00:56:43	200	528.3µs	::1	GET	"/api/v1/db/U7f9C9"
[GIN]	2025/01/06 - 00:56:43	200	0s	::1	GET	"/api/v1/db/BxC11"
[GIN]	2025/01/06 - 00:56:43	200	0s	::1	GET	"/api/v1/db/lgTC15"
[GIN]	2025/01/06 - 00:56:43	200	530.6µs	::1	GET	"/api/v1/db/4WUC8"
[GIN]	2025/01/06 - 00:56:43	200	60.7µs	::1	GET	"/api/v1/db/014bvC14"
[GIN]	2025/01/06 - 00:56:43	200	0s	::1	GET	"/api/v1/db/1sC16"
[GIN]	2025/01/06 - 00:56:43	200	0s	::1	GET	"/api/v1/db/1sC16"
[GIN]	2025/01/06 - 00:56:43	200	0s	::1	GET	"/api/v1/db/lgTC15"
[GIN]	2025/01/06 - 00:56:43	200	0s	::1	GET	"/api/v1/db/014bvC14"
[GIN]	2025/01/06 - 00:56:43	200	0s	::1	GET	"/api/v1/db/ZgAcFC18"
[GIN]	2025/01/06 - 00:56:43	200	0s	::1	GET	"/api/v1/db/4WUC8"
[GIN]	2025/01/06 - 00:56:43	200	0s	::1	GET	"/api/v1/db/KwSC6"
[GIN]	2025/01/06 - 00:56:43	200	0s	::1	GET	"/api/v1/db/CxAC1"

Figura 20 – Amostra do terminal indicando 200 OK para 1000 requisições GET em menos de 2 segundos, sem erros no lado do servidor.

Na Figura 20, observa-se uma parte do resultado gerado pelo *script* Python, onde cada linha corresponde a uma requisição GET enviada ao serviço em um intervalo muito curto (aproximadamente menos de 2 segundos para todas as 1000 chamadas). O **status code** 200 apareceu consistentemente, indicando sucesso na recuperação das informações. Além disso, não foram relatados erros no terminal do servidor, evidenciando que o sistema suportou essa carga de leitura sem falhas aparentes.

Já na Figura 21, é apresentada uma parte do terminal destacando como o *script* Python validou o *payload* retornado para cada requisição. Em cada teste, a chave recebida foi comparada com a esperada. A combinação de **status code** 200 e a correspondência exata dos dados comprova que o serviço retornou as chaves corretamente. Esse processo de verificação indica a robustez do sistema para lidar com alto volume de requisições GET de maneira simultânea ou em

rápida sucessão, sustentando o requisito de **escala e disponibilidade** em cenários de leitura intensa.

5.4.2.6 Teste de Inserção e Recuperação de PDF via Interface Web

Uma última verificação importante consistiu em avaliar a capacidade do sistema de lidar com arquivos PDF grandes, convertendo-os para **Base64** e posteriormente retornando o documento íntegro. O teste foi realizado por meio da aplicação web descrita na Seção ??, evidenciando todo o fluxo de armazenamento e recuperação de um arquivo PDF.

Procedimento:

1. Selecione um PDF local (no caso, `historicoescolarListar (28).pdf`) na interface da aplicação, atribuindo o nome “**historico escolar**” para identificação.
2. Pressione o botão “*Registrar PDF*”, que converte o arquivo para **Base64** e envia a requisição para o banco de dados distribuído (Figura 22).
3. Em seguida, insira o mesmo nome “**historico escolar**” no campo de consulta e pressione “*BAIXAR PDF*” (Figura 23).

Evidência de Teste: - A Figura 22 apresenta o momento em que o arquivo PDF é escolhido e recebe o nome “**historico escolar**”, pronto para ser enviado ao sistema. - Já a Figura 23 ilustra a requisição de recuperação (GET), onde o campo “**nome do PDF desejado**” corresponde exatamente ao nome atribuído anteriormente. A aba *Networking* do Google Chrome mostra o retorno da **string Base64** oriunda do banco de dados, evidenciando a resposta 200 OK. Após a decodificação, o arquivo PDF é baixado e aberto localmente sem qualquer corrupção, confirmando a integridade do processo de armazenamento e recuperação.

```

920: GET key=hClnC12 - SUCESSO - Tempo: 0.00051665s
921: GET key=BxC11 - SUCESSO - Tempo: 0.00052428s
922: GET key=MH3C18 - SUCESSO - Tempo: 0.00053024s
923: GET key=sQA1C13 - SUCESSO - Tempo: 0.00052762s
924: GET key=qSC3 - SUCESSO - Tempo: 0.00058150s
925: GET key=CxAc1 - SUCESSO - Tempo: 0.0006676s
926: GET key=U7F9C9 - SUCESSO - Tempo: 0.00059247s
927: GET key=TxdMR04 - SUCESSO - Tempo: 0.00015879s
928: GET key=LgTC15 - SUCESSO - Tempo: 0.00065827s
929: GET key=qSC3 - SUCESSO - Tempo: 0.00072885s
930: GET key=xt1C5 - SUCESSO - Tempo: 0.00014591s
931: GET key=ZgAcFC18 - SUCESSO - Tempo: 0.0006986s
932: GET key=CxAc1 - SUCESSO - Tempo: 0.0007987s
933: GET key=BxC11 - SUCESSO - Tempo: 0.0008882s
934: GET key=LgTC15 - SUCESSO - Tempo: 0.00061584s
935: GET key=sQA1C13 - SUCESSO - Tempo: 0.00021124s
936: GET key=4MUC8 - SUCESSO - Tempo: 0.00011969s
937: GET key=8L4byC14 - SUCESSO - Tempo: 0.00059986s
938: GET key=4MUC8 - SUCESSO - Tempo: 0.00072551s
939: GET key=ZhC2 - SUCESSO - Tempo: 0.00163031s
940: GET key=E2FC17 - SUCESSO - Tempo: 0.00150585s
941: GET key=BxC11 - SUCESSO - Tempo: 0.00071907s
942: GET key=WrSC6 - SUCESSO - Tempo: 0.00021386s
943: GET key=ZgAcFC18 - SUCESSO - Tempo: 0.00011277s
944: GET key=WrSC6 - SUCESSO - Tempo: 0.00056386s
945: GET key=MH3C18 - SUCESSO - Tempo: 0.0009727s
946: GET key=xt1C5 - SUCESSO - Tempo: 0.00087415s
947: GET key=xt1C5 - SUCESSO - Tempo: 0.00087153s
948: GET key=ZgAcFC18 - SUCESSO - Tempo: 0.0006866s
949: GET key=qSC3 - SUCESSO - Tempo: 0.00073051s
950: GET key=CxAc1 - SUCESSO - Tempo: 0.00080776s
951: GET key=U7F9C9 - SUCESSO - Tempo: 0.00080276s
952: GET key=8L4byC14 - SUCESSO - Tempo: 0.00074410s
953: GET key=1sC16 - SUCESSO - Tempo: 0.00015950s
954: GET key=TxdMR04 - SUCESSO - Tempo: 0.00051355s
955: GET key=CxAc1 - SUCESSO - Tempo: 0.00062609s
956: GET key=8L4byC14 - SUCESSO - Tempo: 0.00106978s
957: GET key=BxC11 - SUCESSO - Tempo: 0.00084162s
958: GET key=4MUC8 - SUCESSO - Tempo: 0.00018764s
959: GET key=4MUC8 - SUCESSO - Tempo: 0.00068212s
960: GET key=8L4byC14 - SUCESSO - Tempo: 0.00104165s
961: GET key=hClnC12 - SUCESSO - Tempo: 0.00134778s
962: GET key=1sC16 - SUCESSO - Tempo: 0.00060021s
963: GET key=ZgAcFC18 - SUCESSO - Tempo: 0.00053883s
964: GET key=hClnC12 - SUCESSO - Tempo: 0.00053024s
965: GET key=BxC11 - SUCESSO - Tempo: 0.00053144s
966: GET key=qSC3 - SUCESSO - Tempo: 0.00052714s
967: GET key=hClnC12 - SUCESSO - Tempo: 0.00060832s
968: GET key=xt1C5 - SUCESSO - Tempo: 0.0007987s
969: GET key=8L4byC14 - SUCESSO - Tempo: 0.0005255s
970: GET key=uHQ7 - SUCESSO - Tempo: 0.00053835s
971: GET key=uHQ7 - SUCESSO - Tempo: 0.00058389s
972: GET key=uHQ7 - SUCESSO - Tempo: 0.00010991s
973: GET key=xt1C5 - SUCESSO - Tempo: 0.00056529s
974: GET key=U7F9C9 - SUCESSO - Tempo: 0.00056529s
975: GET key=1sC16 - SUCESSO - Tempo: 0.00087200s
976: GET key=4MUC8 - SUCESSO - Tempo: 0.00056267s
977: GET key=sQA1C13 - SUCESSO - Tempo: 0.00059891s
978: GET key=sQA1C13 - SUCESSO - Tempo: 0.00016093s
979: GET key=1sC16 - SUCESSO - Tempo: 0.00050640s
980: GET key=E2FC17 - SUCESSO - Tempo: 0.0006485s
981: GET key=1sC16 - SUCESSO - Tempo: 0.00056124s
982: GET key=uHQ7 - SUCESSO - Tempo: 0.00058317s
983: GET key=xt1C5 - SUCESSO - Tempo: 0.00060821s
984: GET key=U7F9C9 - SUCESSO - Tempo: 0.00052857s
985: GET key=BxC11 - SUCESSO - Tempo: 0.00147200s
986: GET key=LgTC15 - SUCESSO - Tempo: 0.00112534s
987: GET key=4MUC8 - SUCESSO - Tempo: 0.00053072s
988: GET key=8L4byC14 - SUCESSO - Tempo: 0.0005858s
989: GET key=1sC16 - SUCESSO - Tempo: 0.00087248s
990: GET key=1sC16 - SUCESSO - Tempo: 0.00063443s
991: GET key=LgTC15 - SUCESSO - Tempo: 0.00026870s
992: GET key=8L4byC14 - SUCESSO - Tempo: 0.00011539s
993: GET key=ZgAcFC18 - SUCESSO - Tempo: 0.0008750s
994: GET key=4MUC8 - SUCESSO - Tempo: 0.00050545s
995: GET key=WrSC6 - SUCESSO - Tempo: 0.00055146s
996: GET key=CxAc1 - SUCESSO - Tempo: 0.00009131s
997: GET key=hClnC12 - SUCESSO - Tempo: 0.0005984s
998: GET key=qSC3 - SUCESSO - Tempo: 0.00059867s
999: GET key=TxdMR04 - SUCESSO - Tempo: 0.00020742s
1000: GET key=uHQ7 - SUCESSO - Tempo: 0.00061131s

```

Figura 21 – Trecho do terminal mostrando correspondência exata entre a chave solicitada e a chave retornada em cada requisição.

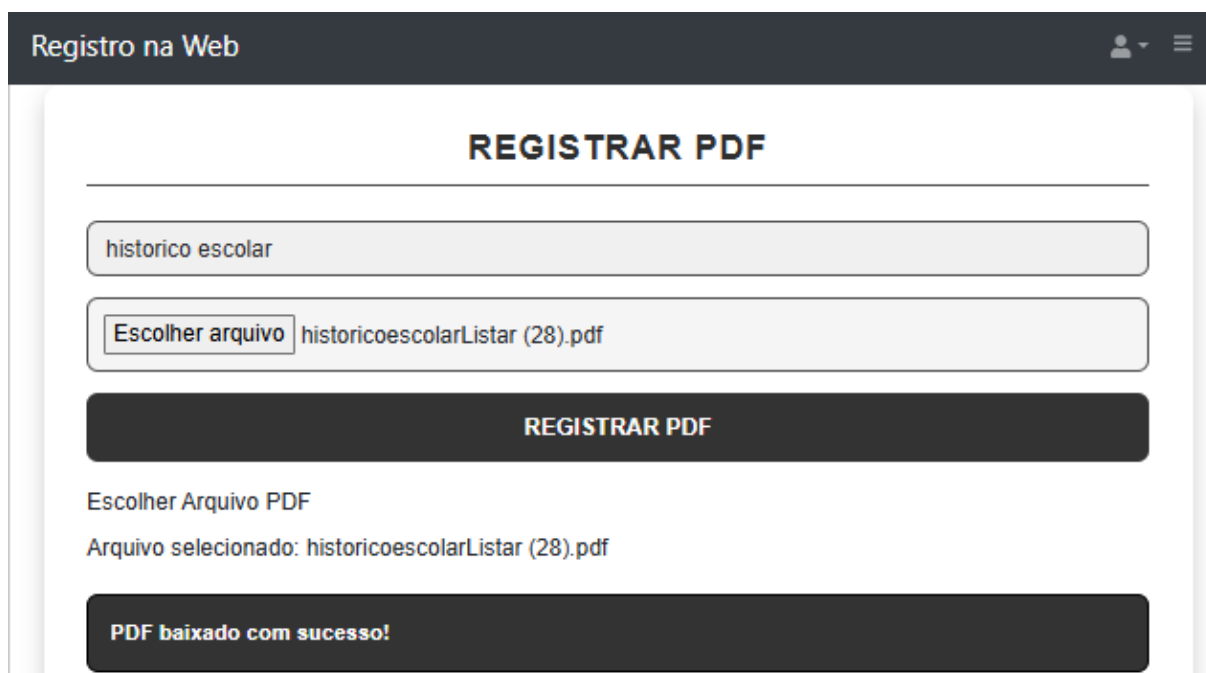


Figura 22 – Registro de PDF na aplicação web, atribuindo o nome “*historico escolar*” ao arquivo `historicoescolarListar (28).pdf` antes de pressionar “Registrar PDF”.

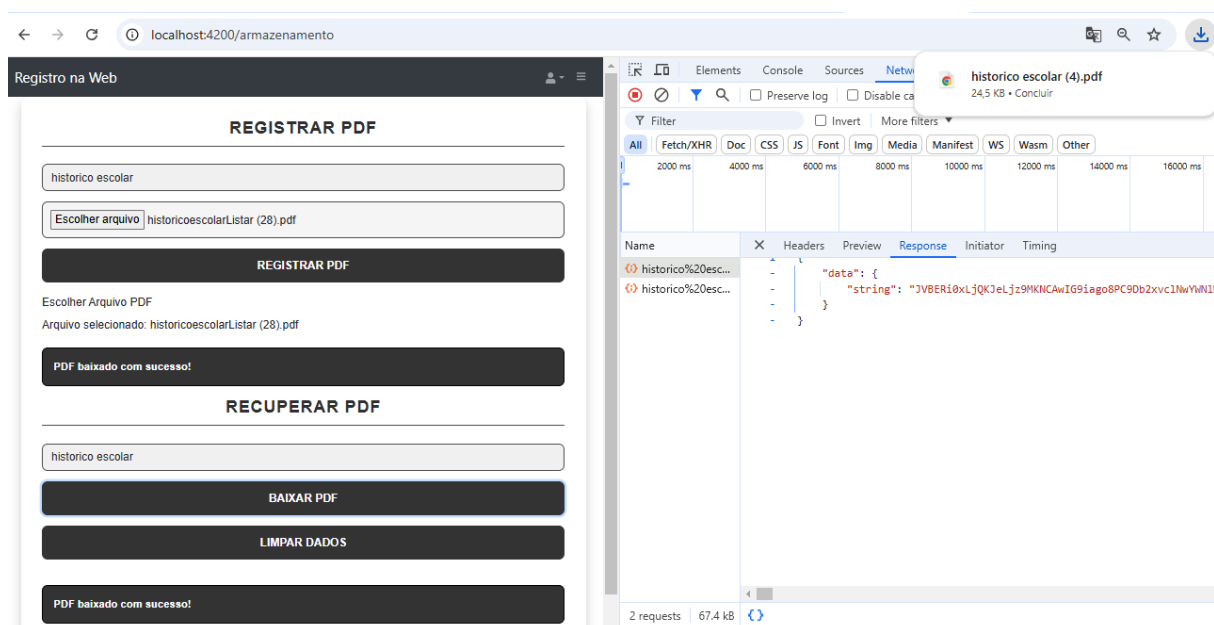


Figura 23 – Visualização da aplicação web ao recuperar o PDF. O campo “nome do PDF desejado” contém “*historico escolar*”, e o botão “*BAIXAR PDF*” foi acionado. A aba *Networking* do navegador exibe a resposta em Base64, confirmando o download bem-sucedido do arquivo.

6

Considerações Finais

Este capítulo apresenta uma análise global dos resultados obtidos e as lições aprendidas com o desenvolvimento do sistema distribuído de banco de dados chave-valor. Serão discutidas, de forma detalhada, as principais conclusões a respeito da escalabilidade, tolerância a falhas, consistência, e utilização prática do sistema, evidenciando suas limitações e potenciais aplicações.

6.1 Análise Geral do Desempenho e da Escalabilidade

Uma das constatações mais importantes refere-se ao comportamento do sistema perante diferentes padrões de carga. Conforme descrito na subseção ***Limitação de Taxa e Erros em Altas Taxas de Inserção*** (ver Capítulo de Desenvolvimento), foi verificado que:

- O nó líder não consegue processar de forma confiável **altas taxas de inserção** (por exemplo, 1000 POST em 10 segundos), gerando um acúmulo de requisições pendentes e, eventualmente, retornando erros 500 Internal Server Error.
- Ao impor, do lado do requerente uma restrição de “**1 inserção por segundo**”, o sistema consegue lidar de forma previsível com o fluxo de POST, evitando falhas na replicação e garantindo a estabilidade operacional.

Esse cenário se explica pelo fato de o nó líder, em arquiteturas baseadas no algoritmo Raft, necessitar propagar cada escrita aos seguidores para manter a consistência dos logs replicados. Quando a cadência de novas inserções ultrapassa a capacidade de processamento e de sincronização, as requisições acumulam-se rapidamente.

Em contrapartida, a seção ***Teste de Carga com 1000 Requisições GET*** mostrou que o sistema pode responder de forma extremamente rápida a operações de leitura: **500 respostas por segundo**, sem apresentar erros no terminal. Esse resultado destaca o fato de que o sistema distribui bem as requisições de leitura entre os nós e não sofre tanto com o overhead de replicação ao servir GET, fazendo dele uma alternativa apropriada para aplicações em que a maioria das operações consista em consultas.

6.2 Desafios encontrados

Durante os testes realizados entre setembro e dezembro de 2024, houve a ocorrência de um erro aleatório relacionado ao processo de eleição de líder no cluster. Esse problema aconteceu em dois momentos distintos, mas não foi possível replicá-lo de maneira consistente. Em ambos os casos, os nós não conseguiram entrar em consenso sobre qual deveria ser o líder do cluster, resultando em uma falha na coordenação das operações.

Como efeito desse impasse, nenhuma operação de manutenção de dados (POST, GET, DELETE, PUT) foi executada com sucesso, comprometendo completamente a funcionalidade do sistema. Para resolver essa situação, foi necessário reiniciar o serviço, o que permitiu ao sistema retomar o funcionamento adequado, restabelecendo o consenso entre os nós através de uma nova eleição e a continuidade das operações. Este evento aponta para uma área que merece atenção futura, a fim de melhorar a robustez do processo de eleição e evitar que o sistema entre em um estado inconsistente, interrompendo todas as operações do banco de dados.

Embora os testes realizados evidenciem a capacidade do sistema em situações controladas, verificou-se que a comprovação plena da tolerância a falhas e da escalabilidade encontra limitações. Os experimentos, conduzidos com clusters de pequena escala (3 a 5 nós), não abrangeram cenários de falhas simultâneas ou partições de rede prolongadas, o que restringe a demonstração da robustez do sistema. Ademais, a ausência de comparativos diretos com soluções consolidadas no mercado dificulta a aferição inequívoca dos ganhos de desempenho obtidos. Tais limitações apontam para a necessidade de estudos futuros com simulações mais abrangentes e testes em ambientes de produção, a fim de confirmar a confiabilidade e a capacidade de expansão do sistema em contextos de alta criticidade.

6.3 Validação de Integridade e Tolerância a Falhas

Através dos diversos testes, ficou claro que:

- **Integridade de Arquivos Binários (PDFs):** A aplicação web exemplificou a conversão de arquivos PDF para Base64 e seu posterior armazenamento no banco de dados distribuído. Uma vez recuperados, esses arquivos foram decodificados sem qualquer corrupção de conteúdo. Essa funcionalidade cumpre o requisito de integridade de dados e ilustra como mesmo arquivos volumosos podem ser manipulados de forma confiável.
- **Mecanismos de Recuperação Raft:** Quando líder e seguidores enfrentam cenários de falhas ou eleição de novos líderes, conforme descrito nos testes de

consistência e eleição, o protocolo Raft demonstrou capacidade de retomar a operação sem comprometer os dados já armazenados. O sistema se mostrou resiliente a paradas abruptas e conseguiu se reorganizar, elegendo um novo líder e preservando a coerência global.

6.4 Conclusão Geral

No decorrer deste trabalho, foi desenvolvida e avaliada uma solução de banco de dados chave-valor distribuído, capaz de manipular grandes volumes de dados e arquivos binários sem corrompê-los, bem como resistir a falhas de nós por meio de um algoritmo de consenso (Raft). A **aplicação web de PDF** validou a confiabilidade do armazenamento, demonstrando a integridade dos documentos decodificados. Observou-se, porém, que o sistema enfrenta dificuldades frente a **picos de inserção** extremamente intensos, exigindo a adoção de técnicas de controle de taxa e escalabilidade mais agressiva para contorná-los. Em contrapartida, as **operações de leitura (GET)** mostraram-se altamente eficientes, sustentando centenas de respostas por segundo sem falhas. Isso ressalta a vocação do sistema para situações de **muitas leituras e poucas inserções**, típico de várias aplicações de cache, indexação ou relatórios.

Dessa maneira, os objetivos definidos inicialmente foram atendidos na maior parte, confirmando o potencial do banco de dados para uso em cenários de leitura intensiva. As descobertas sobre limites de inserção apontam caminhos futuros de pesquisa e otimização, tornando o sistema ainda mais versátil e robusto para aplicações de maior escala.

6.5 Perspectivas de Continuidade

Este trabalho representa uma base sólida para futuras melhorias e adaptações que podem expandir sua funcionalidade, performance e aplicabilidade. A seguir, são sugeridas algumas direções para a continuidade do projeto:

- **Implementação em Nuvem:** Uma evolução natural seria a migração do sistema para um ambiente de computação em nuvem, aproveitando plataformas como AWS, Google Cloud ou Azure. Isso permitiria explorar virtualização e orquestradores de containers, como Kubernetes, para escalabilidade automática (*autoscaling*), maior resiliência e facilidade de gerenciamento de recursos.
- **Refinamento da Arquitetura:** As camadas da aplicação podem ser reestruturadas para separar mais claramente as responsabilidades. Uma proposta seria isolar a lógica da API REST e do balanceamento de carga, tornando o sistema

mais modular e facilitando sua manutenção e evolução. Isso também abriria espaço para o uso de proxies reversos dedicados, como NGINX ou Envoy, para otimizar o desempenho e a segurança.

– **Extensão de Aplicações:**

- * **Cache para Aplicações Web:** A solução pode ser adaptada como um sistema de cache para aplicações web de alto desempenho, armazenando dados temporários e reduzindo o tempo de resposta para requisições frequentes.
- * **Base para Banco de Dados SQL:** A estrutura distribuída e escalável do banco de dados chave-valor poderia ser expandida para incluir funcionalidades relacionais, resultando em um banco de dados SQL distribuído, que combina alta disponibilidade com suporte a consultas complexas.

– **Integração com Tecnologias Emergentes:** O sistema pode ser integrado a novas tecnologias, como análises de dados em tempo real ou sistemas de *machine learning*, aproveitando sua arquitetura distribuída para lidar com grandes volumes de dados e operações simultâneas.

– **Melhoria na Observabilidade:** Ampliar os recursos de monitoramento e diagnóstico do sistema, com o uso de ferramentas como Prometheus e Grafana, poderia melhorar a visibilidade de desempenho, falhas e uso de recursos.

Essas direções não apenas aumentariam a robustez e a versatilidade do sistema, como também o tornariam aplicável a um leque mais amplo de cenários reais, promovendo sua adoção em ambientes corporativos e acadêmicos.

Referências

- Amazon Web Services. *O que é um banco de dados de chave-valor?* 2024. Disponível em: <<https://aws.amazon.com/pt/nosql/key-value/>>. Citado na página 24.
- BREWER, E. A.; FOX, A. Lessons from giant-scale services. *IEEE Internet Computing*, v. 3, n. 4, p. 46–55, 1999. Citado na página 15.
- CARLSON, J. L. *Redis in Action*. [S.l.]: Manning Publications, 2013. Citado na página 25.
- CHANG, F. et al. Bigtable: A distributed storage system for structured data. In: *ACM Transactions on Computer Systems*. [S.l.: s.n.], 2008. p. 4–7. Citado 2 vezes nas páginas 12 e 13.
- COMMUNITY gRPC W. *gRPC-Web: A JavaScript Client Library for gRPC*. 2024. Framework desenvolvido pela comunidade para suporte parcial ao gRPC em navegadores. Disponível em: <<https://github.com/grpc/grpc-web>>. Citado na página 17.
- DECANDIA, G. et al. Dynamo: amazon’s highly available key-value store. In: *Proceedings of the 21st ACM Symposium on Operating Systems Principles 2007, SOSOP 2007, Stevenson, Washington, USA, October 14-17, 2007*. [S.l.: s.n.], 2007. p. 205–220. Citado na página 11.
- HUNT, P. et al. Zookeeper: Wait-free coordination for internet-scale systems. In: *Proceedings of the USENIX Annual Technical Conference*. [S.l.: s.n.], 2010. p. 145–158. Citado na página 16.
- IMASTERS. *Introdução ao Redis, o NoSQL chave-valor mais famoso*. 2024. Disponível em: <<https://imasters.com.br/banco-de-dados/introducao-ao-redis-o-nosql-chave-valor-mais-famoso>>. Citado na página 25.
- LAKSHMAN, A.; MALIK, P. Cassandra: A decentralized structured storage system. In: *Proceedings of the 3rd ACM Symposium on Cloud Computing*. [S.l.: s.n.], 2010. p. 1–12. Citado 5 vezes nas páginas 12, 13, 25, 28 e 30.
- LAMPORT, L. *Generalized Consensus and Paxos*. [S.l.], 2005. Disponível em: <<https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/tr-2005-33.pdf>>. Citado 2 vezes nas páginas 16 e 29.
- MEDJAHED, B.; OUZZANI, M.; ELMAGARMID, A. Generalization of acid properties. *Cyber Center Publications*, n. 97, 2009. Disponível em: <<http://docs.lib.purdue.edu/ccpubs/97>>. Citado na página 15.
- NAPOLEON. *O que é: Key-Value Database*. 2024. Disponível em: <<https://napoleon.com.br/glossario/o-que-e-key-value-database/>>. Citado na página 24.
- ONGARO, D.; OUSTERHOUT, J. In search of an understandable consensus algorithm. In: *Proceedings of the USENIX Annual Technical Conference*. [S.l.: s.n.], 2014. p. 305–320. Citado 4 vezes nas páginas 16, 28, 29 e 30.

STALLINGS, W. *Operating Systems: Internals and Design Principles*. [S.l.]: Pearson, 2015. 409 p. ISBN 978-0-13-380591-8. Citado na página 17.

Uber Technologies, Inc. *Zap Logger*. 2024. Acesso em: 09 dez. 2024. Disponível em: <<https://github.com/uber-go/zap>>. Citado na página 33.

- BREWER, Eric A.; FOX, Armando. ***Lessons from giant-scale services***. *IEEE Internet Computing*, v. 3, n. 4, p. 46-55, 1999. Este artigo introduz o teorema CAP, descrevendo as características de consistência, disponibilidade e tolerância a partições, relevantes para sistemas distribuídos. Citado na página 15.
- ONGARO, Diego; OUSTERHOUT, John. ***In search of an understandable consensus algorithm***. *Proceedings of the USENIX Annual Technical Conference*, p. 305-320, 2014. Este trabalho apresenta o algoritmo Raft, um protocolo de consenso que visa garantir a consistência em sistemas distribuídos de maneira simplificada e compreensível. Citado 4 vezes nas páginas 16, 28, 29 e 30.
- HUNT, Patrick et al. ***ZooKeeper: Wait-free coordination for Internet-scale systems***. *Proceedings of the USENIX Annual Technical Conference*, p. 145-158, 2010. Este estudo discute o Zab, algoritmo de consenso utilizado pelo ZooKeeper para coordenar sistemas distribuídos, com ênfase na consistência e tolerância a falhas. Citado na página 16.